University of Bergen
Department of Informatics

Performance of Distributed and Shared Memory Parallel Sparse Matrix Vector Multiplication

Author: Kristian Sørdal
Supervisor: Johannes Langguth
Co-Supervisor: Fredrik Manne

UNIVERSITETET I BERGEN
Fakultet for naturvitenskap og teknologi

June, 2025

To err is human; but to really foul things up requires a computer.

Paul R. Ehrlich

Abstract

Sparse-Matrix Vector Multiplication (SpMV) is a computational kernel that arise in many applications of scientific computing. It is a kernel that has an inherently low computational intensity, and its performance is therefore bound by the memory bandwidth of the system it is run on. The performance of SpMV can be improved through the use of both shared and distributed memory parallelization. Parallelizing SpMV in a distributed memory environment requires careful attention to managing the communication patterns which arise when a matrix is distributed on multiple nodes, to reduce the memory traffic across the interconnects.

This thesis aims to investigate the impact that five increasingly selective communication strategies have on the performance of SpMV. To achieve this, these communication strategies have been tested on a set of parallel hardware that includes both dual-socketed and single-socketed compute nodes. The configurations used on these compute nodes include both single-node non-shared memory, and a selection of multi-node shared and distributed memory configurations. In the experiments, 100 iterations of SpMV were executed for each communication strategy.

The findings from the experiments performed in this thesis show that each communication strategy indeed yields lower communication volumes. The findings show that lower communication volume on a single compute node consistently yields better performance. They also show that in a multi-node environment, performance is not solely dictated by the communication volume, but also by other factors inherent to the network characteristics and hardware of the compute nodes.

Acknowledgements

I would like to express my gratitude to my supervisor Johannes Langguth, for the helpful guidance, valuable discussions and insighful feedback he provided throughout the process of writing this thesis.

I would also like to thank my fellow students and friends in the Algorithms study hall, for all the fun times, frustrations shared, discussions had, and general shenanigans every day.

Finally, to my family for their unwaivering support and encouragement as a student, and to my friends, with whom I've spent much of my free time with.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Sparse-Matrix Vector Multiplication (SpMV) is a fundamental computational kernel in many scientific computing applications. It appears in a broad range of domains, such as iterative solvers for partial differntial equations, machine learning, and when simulating physical systems. As a result, optimizing the performance of SpMV, both on CPUs and GPUs remains a widely researched topic. SpMV lends itself nicely to paralellization, but it is a kernel with an inherently low computational intensity, and its performance is therefore memory bound.

This thesis aims to investigate the impact that different communication strategies have on the performance of SpMV. The experiments conducted in this thesis have been conducted using a distributed and shared memory hybrid parallel environment. To facilitate inter node communication, one MPI rank has been placed on each compute node (or on each socket if the compute node is dual-socketed), and shared memory paralleization with OpenMP has been used for intra node communication.

Four distinct communication strategies have been tested, with an additional fifth strategy that is fully memory scalable. These communication strategies successively become more sophisticated. The simplest being a strategy which communicates the entire local part of the dense x vector to each other rank. Then two separator based strategies, which involves communicating only the set of boundary elements that induce data-dependencies between nodes. Finally, the most sophisticated communication strategy involves communicating only the elements that induce data-dependencies to the node where this data-dependency occurs.

Each communication strategy is tested on a set of modern multicore architectures available on Simula's *Exploration of Experimental Exascale computing* (eX^3), using a set of well structured sparse matrices.

Chapter 2

Theory

2.1 Sparse Matrices

There are many different opinions on what is considered a sparse matrix, and there exists no definition that captures what a sparse matrix is. For this thesis, a matrix is considered sparse if it is worthwhile to treat nonzeros differently than zeros. Usually this means that for an $n \times n$ matrix, each row has a constant ($\mathcal{O}(1)$) number of nonzero entries per row. Or in other words, a $n \times n$ matrix is sparse if it has $\mathcal{O}(n)$ nonzero entires.

Given a sparse matrix A, an input vector x and the resulting output vector y, Matrix Vector Multiplication is defined as

$$y = Ax$$

where the y_i is given by $y_i = \sum_{j=1}^{n} a_{ij} x_j$.

2.2 Sparse Matrix Vector Multiplication

Sparse-Matrix Vector Multiplication (SpMV) can benefit from paralellization, both using shared memory and distributed memory, or a hybrid of both. Despite this, it is notoriosly difficult to optimize, sequentially and in a parallel setting. This is especially true for the large matrices that appear when solving problems in scientific computing.

As mentioned in the introduction, SpMV has an inherently low computational intensity, and its performance is therfore memory bound. This means that the memory bandwidth of the system becomes the primary bottleneck when scaling to multiple compute nodes. It becomes important

to carefully manage data dependencies between processes, as performance is directly correlated with the communication volume across the interconnects between nodes. This holds true regardless of the hardware that is used, and most of the challenges discussed in this thesis need to be taken into consideration when implementing SpMV on CPUs, GPUs, or other types of architecture.

2.3 Computational Intensity

The *computational intensity* of an operation is a key concept in performance analysis, and is defined as the ratio of the number of FLOPS to the number of memory accesses.

$$\text{Computational intensity} = \frac{\text{FLOPS}}{\text{Memory accesses}} \tag{2.1}$$

This metric helps determine whether a computation is *compute bound* or *memory bound*. Operations with high computational intensity implies that it can benefit from employing faster processors, as it performs a higher number of calculation per byte of memory read. Operations with lower computational intensity implies that the operation in memory bound, and thus employing faster processors wont necessarily yield higher performance, as the limiting factor is not the speed at which the calculations are performed, but rather the speed at which the required memory to perform the operation can be accessed at.

2.4 Latency Numbers

The execution time of computational operations can vary significantly, and it is important to have some understanding of the latency times associated with typical operations. Referencing these latency numbers can be valuable when interpreting benchmark results and the performance of different programs.

Table 2.1: Latency numbers for common operation, adapted from [14].

Operation	**Time [ns]**
L1 cache reference	1
Branch misprediction	3
L2 cache reference	4
Mutex lock/unlock	17
Main memory reference	100
Compress 1K bytes with Zippy	2000
Send 2kB over 10 Gbps network	1600
Send 1K bytes over 1 Gbps network	10 000
Read 4K bytes randomly from SSD*	20 000
Round trip within same datacenter	500 000
Read 1MB sequentially from memory	1 000 000
Disk seek	10 000 000
Read 1MB sequentially form disk	10 000 000
TCP packet round trip between continents	150 000 000

2.5 Parallel Architectures

There are two main architectures used in the parallel computing industry: Shared Memory Architecture and Distributed Memory Architecture. The following sections gives an overview of the key difference between the two.

2.5.1 Shared Memory Architecture

On a system with shared memory architecture, every processing unit (PU) (often referred to as threads) have access to the same memory. Although each thread maintains its own private cache for performance, reads and writes occur in the same shared memory pool. To guarantee correctness, the system must enforce *cache coherence*. Cache coherence means that each read must observe the most recent write, regardless of which cache holds that value. As the on-chip memory controller can perform only one write per clock cycle, if a thread requires a value stored in another threads cache, what is known as the *cache coherence protocol* ensures that the thread obtains the correct value. Cache coherence is crucial in a shared memory setting in order to ensure correct program behaviour and avoid race conditions [16].

2.5.2 Distributed Memory Architecture

On systems with distributed memory architectures, the constituents of the system have their own local memory, not accessible by the others. The system is comprised typically of either a processor in a chip, or a compute node in a cluster, and is usually referred to as a *rank*. When a rank needs to access memory from another rank, explicit communication of the data stored at that memory address needs to occur, and happens through whichever network the ranks are connected with (adapted from [12]). Figure 2.1 illustrates the difference between shared and distributed memory architectures.

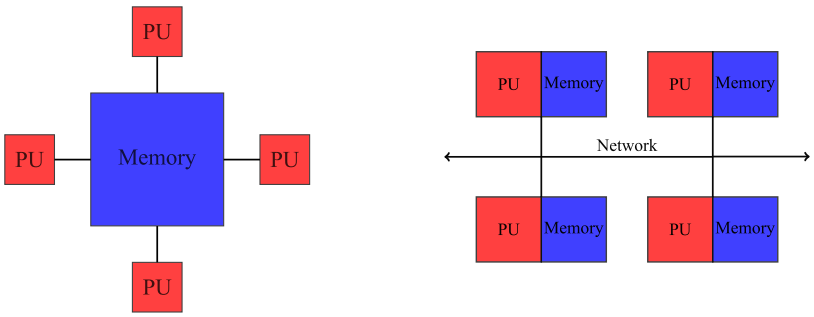


Figure 2.1: Shared and Distributed Memory Architecture (adapted from [11]).

2.5.3 Non-Uniform Memory Access

On multiprocessor systems, Non-Uniform Memory Access (NUMA) memory architecture that lays out memory in such a way that the memory access latency depends on the physical distance between the processor core, and the location of the memory being accessed. On these systems each processor has its own local memory, which cores on that processor can access much faster than memory local to another processor. Figure 2.2 illustrates a system with two NUMA nodes, and if a core on node 0 needs to access memory on node 1, it needs to travel through the interconnect that connects the two nodes, which incurs higher latency (see Table 2.1).

Single chips can also partition their cores into multiple NUMA domains, as an example, a 32-core chip may have 4 NUMA domains where every domain contains eight nodes each. Respecting the NUMA domains is particularly important if performance is the goal, and can be achieved using tools such as `numactl`.

It is important to note that NUMA refers specifically to the organization of the main memory, i.e. the DRAM, and not to the on chip caches implemented using SRAM.

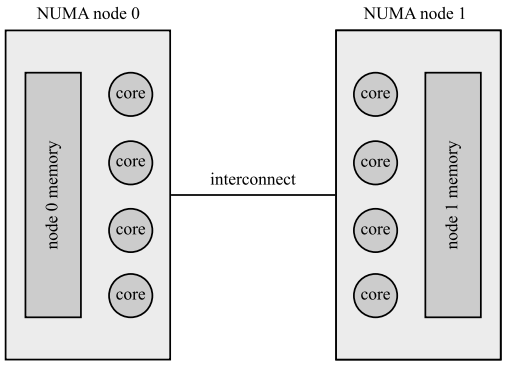


Figure 2.2: System with two NUMA nodes and eight processors (4 per NUMA node) (adapted from [10]).

2.5.4 First-Touch Policy

An important implication of the NUMA memory architecture is how and where memory pages are allocated. This is goverend by what is known as the First-Touch Policy, which is the default memory allocation policy Linux and other operating systems. According to this policy, a memory page is allocated in the memory that is closest to the thread that first requires this memory page. As an example, using Figure 2.2 as reference, if a core on NUMA node 0 is the first to access some memory page, it will be allocated in node 0's memory.

2.5.5 Emerging Architectures

In the ever evolving world of computer chip manufacturing, there has in the past years emerged a new trend in the field of architecture design. This

involves packing thousands of small processor cores into a single device, where each core has its own local memory, and no device level shared memory. An example of such a processor is the relatively new Cerebras Wafer-Scale Engine (WSE-3), which is the largest chip ever built. With a spec sheet of 4 trillion transistors, 900 000 cores, memory bandwidth of 21PB/s, and 44 GB of on-wafer memory (see [7]). Another example is Graphcores Colossus MK2 GC200 IPU (Intelligence Processing Unit). Although not as big as WSE-3, it still contains 59.4 billion transistors and has 1472 cores. Such architectures necessitate a careful distribution of relevant data across each processor, and is where memory scalable communication strategies are beneficial.

Chapter 3

Background

3.1 CSR Storage Format

CSR (Compressed Sparse Row) is the most widely used storage format for sparse matrices. As its name suggests, it compresses the amount of memory used to store a matrix without loss of information. It does so by utilizing three vectors `row_ptr`, `col_idx` and `values`. Figure 3.1 shows an example of a matrix stored in CSR format.

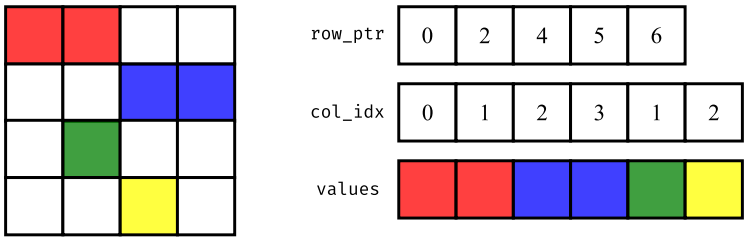


Figure 3.1: Matrix represented in the CSR format (adapted from [6]).

For an $n \times n$ matrix stored in the CSR format, the vector `row_ptr` is of length $n+1$. The i^{th} entry in this vector indicates the starting index in the vectors `col_ptr` and `values` of the i^{th} row. The vectors `col_idx` and `values` stores the column indices and values of the entries in the matrix.

The column indices for the nonzero entries in the i^{th} row are given by the range from `col_idx[row_ptr[`i`]]` to `col_idx[row_ptr[`$i+1$`]]`, and similarly for the `values` vector.

Throughout the remainder of this thesis, we operate under the assumption that all matrices are represented in CSR format, unless explicitly noted otherwise.

3.2 Other storage formats

While the CSR format is the most widely used storage format for sparse matrices, there exists sever other storage formats, each with its own benefits and drawbacks. In this section, some of these formats are briefly presented.

3.2.1 CSC Format

The Compressed Sparse Column (CSC) format is similar to the CSR format, with the difference being that the columns are compressed instead of the rows. In the context of SpMV and other operations on matrices, CSC can be used if operation on the transpose, i.e. A^T, is desirable. For operations on A, this format is less ideal, as cache hit rates will suffer as iteration will be done in a column-major order, instead of a row-major order.

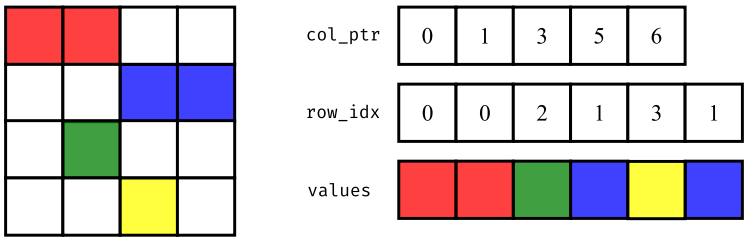


Figure 3.2: Matrix represented in the CSC format.

3.2.2 COO Format

The Coordinate List (COO) format stores the matrix as a set of triples on the form i, j, x, where i is the row index, j is the column index, and x is the value stored at (i, j) in the matrix. In this format all zero entries are ignored.

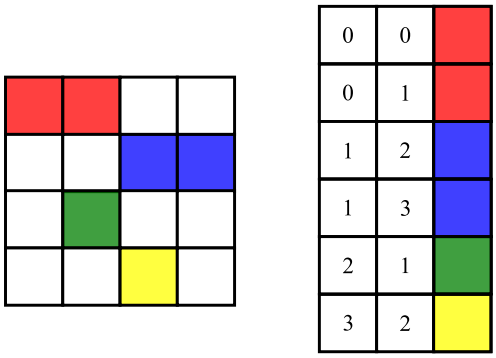


Figure 3.3: Example Matrix represented in the COO format.

3.2.3 ELLPack Format

The ELLPack format stores a matrix using two tables, these being the `Data` and `Indices` tables. These tables have dimensions $N \times K$, where N is the number of rows in the matrix, and K is the number of nonzeros in the row with the largest amount of nonzeros. The i^{th} row of the `Data` and `Indices` tables store the values and columns indices of the nonzeros in the i^{th} row, respectively. For rows where the number of nonzeros is less than K, the rest of the entries in these tables are padded with zeros (adapted from [20]).

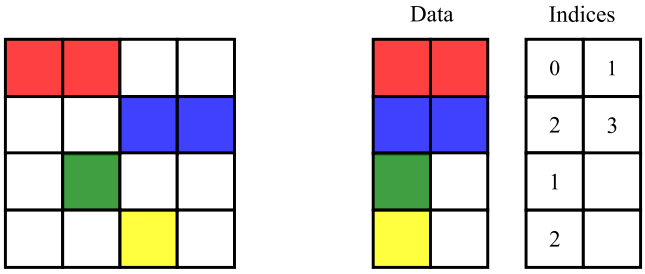


Figure 3.4: Matrix Represented in the ELLPACK format (adapted from [20]).

3.3 Sequential SpMV

A sequential implementation of SpMV on a matrix stored in the CSR format can be implemented in the following manner:

Algorithm 1: Sequential CSR-based SpMV

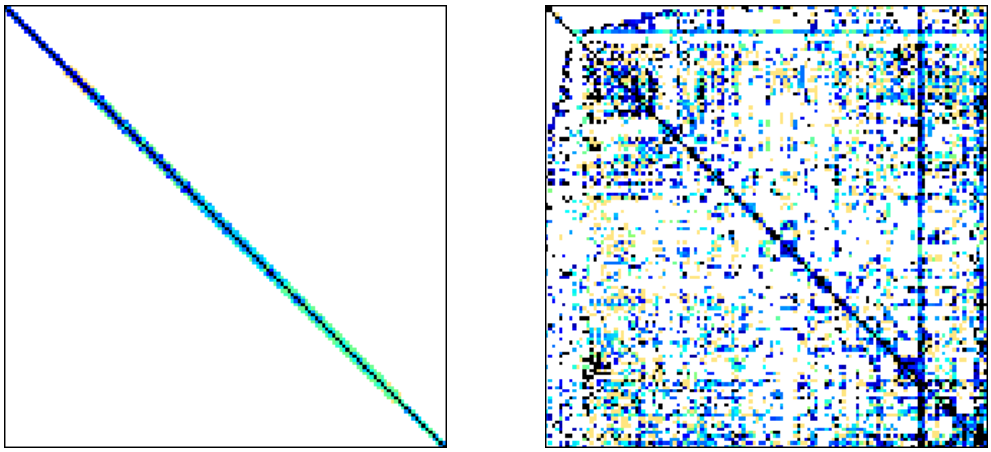
Input : $rowPtr, colIdx, values, x$
Output : y

for $i \leftarrow 0$ **to** n **do**
    sum $\leftarrow 0$
    for $j \leftarrow rowPtr[i]$ **to** $rowPtr[i+1]$ **do**
        sum $=$ sum $+ values[j] \cdot x[colIdx[j]]$
    $y[i] \leftarrow$ sum

From Algorithm 3.3, computing the interim value of `sum` requires two floating point operations (FLOPs). As this is done once for every nonzero in the matrix, each nonzero results in two flops per iteration of SpMV. For SpMV on well-structured matrices such as those similar to (a) in Figure 3.5, each nonzero requires approximately 12 bytes to be read from memory, in order to compute the interim value of `sum`. This may appear inconsistent with the memory access patterns illustrated in the algorithm,

where two doubles and one integer are read per nonzero, corresponding to 20 bytes. This discrepancy stems from the caching behaviours of the input vector x, as after an entry is initially read, it is frequently reused, and thus remains in cache. This means that on average, only one double and one integer is read per nonzero, which corresponds to 12 bytes.

In contrast, for highly unstructured matrices, such as the one seen in (b) in Figure 3.5, memory access patterns can be more irregular. In the worst case scenario, each nonzero triggers the read of an entire cache line of 64 bytes, and only uses a single value stored in this cache line. This can result in $64 + 12 = 76$ bytes read per nonzero.



(a) Cube_Coup_dt0 (b) shermanACd

Figure 3.5: Well structured (a) and poorly structured (b) matrices. (Matrix pattern images collected from the University of Florida Sparse Matrix Collection [4], accessed via the SuiteSparse Matrix Collection Website Interface [9].)

3.4 Shared Memory SpMV

SpMV can be parallelized using the OpenMP directive `#pragma omp parallel for`. By default, this tells OpenMP to use `static` scheduling when parallelizing the outer iteration loop. When static scheduling is used, the span of the iteration that each thread will execute is precomputed, and stays static, as the naming suggests. There are other scheduling options, such as `dynamic` and `guided`, which will be discussed in later sections.

An implementation of shared memory SpMV is outlined below.

Algorithm 2: Shared Memory CSR-based SpMV

Input : $rowPtr, colIdx, values, x$
Output : y

```
#pragma omp parallel for
for i ← 0 to n do
    sum ← 0
    for j ← rowPtr[i] to rowPtr[i+1] do
        sum = sum + values[j] · x[colIdx[j]]
    y[i] ← sum
```

3.4.1 Scheduling options

As seen in Algorithm 2, the outer loop is parallelized, which translates to dividing the rows of the matrix evenly among the threads. This work fine for well structured matrices, but for matrices with dense rows, such as the matrix shown in Figure 3.6, we obtain large imbalances in the computational load for each thread, which impacts performance. This is because the next iteration of SpMV cannot start until all threads have finished their computations on the current iteration, which means that threads will be idle while waiting for the slowest thread to finish its workload.

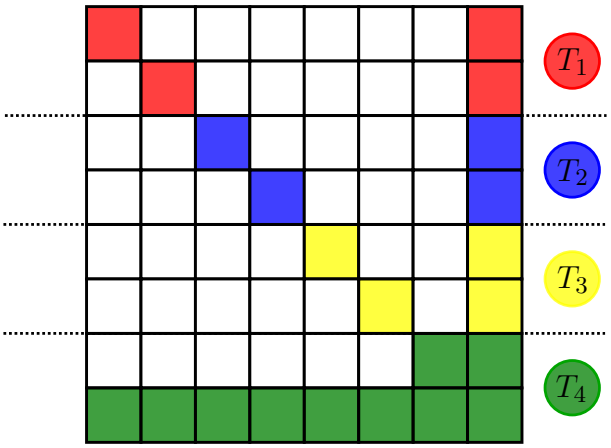


Figure 3.6: Row distribution among threads under static scheduling.

3.4.2 Dynamic Scheduling

Dynamic scheduling in OpenMP involves precomputing the range of iterations without assigning specific iteration subsets to individual threads in advance. Under static scheduling, if thread A receives sparse rows and thread B receives dense rows, thread A will become idle prematurely while thread B remains computationally engaged. In contrast, dynamic scheduling allocates iterations at runtime to threads as they become available, thus potentially balancing the computational load more effectively, particularly when iteration workloads vary significantly.

At first glance, dynamic scheduling appears to resolve the workload imbalance inherent in static scheduling, even though it comes with more overhead. While this assumption holds true on smaller systems, such as personal laptops equipped with fewer physical cores and a single processor socket, it does not readily apply to larger, dual-socket systems utilized for the experiments conducted in this thesis. Here, the first-touch memory policy becomes particularly significant.

3.4.3 Performance Implications of the First-Touch Policy

As shown in Table 2.1, accesses to main memory are significantly slower than accesses to local cache levels such as L1 or L2. On NUMA architectures, this disparity is further amplified when a thread must access memory located on a remote NUMA node. In dual-socket configurations, for example, such accesses require communication across the inter-socket interconnect. Using dynamic or guided scheduling, threads can be assigned iterations that requires memory localized on a different NUMA node, which in turn introduces additional latency and can lead to congestion of the memory bandwidth through the interconnect.

3.5 Distributed Memory SpMV

In distributed memory systems, the matrix, and consequently the computational workload, is distributed across multiple ranks. Distributing the matrix is achieved using a partitioner, which give a partition indicating which ranks are responsible for which part of the matrix. As discussed in Section 2.5.2, each ranks memory is inaccessible to other ranks, and any data-dependency that occurs between ranks must be resolved through the usage of explicit communication of that data. Most commonly this is

achieved through the usage of the Message Passing Interface (MPI). For distributed sparse matrix-vector multiplication (SpMV), the computation proceeds similarly to the shared memory case (algorithm 2). The sparse matrix is divided across nodes, and each node computes its local segment of the output vector y. At the end of each iteration, partial results are assembled to produce the global vector, which necessitates communication to resolve data-dependencies.

While the thread scheduling and memory locality issues discussed in 3.4.1 are still encountered, distributed memory systems introduce the additional challenge of managing communication volume. As shown in Table 2.1, inter-node communication incur much higher latencies than that of memory accesses. Therefore, minmizing the total communication volume per iteration of SpMV can have a large impact on the overall performance.

3.5.1 Load Balancing and Graph Partitioning

It is important that the partitioner yields partitions that are roughly equal in size. But, even with a well balanced partition, managing the data-dependencies between ranks is important. A good partitioner is one that finds a balance between creating parts that are similarly sized, but also reduces the amount of data-dependencies between ranks. This is a problem that graph partitioners attempt to solve.

The graph partitioning problem involes dividing an undirected graph $G(V, E)$, where V is the set of vertices and E is the set of edges into K disjoint subsets (or partitions) such that each subset contains a comparable amount of vertices, and the number of edges connecting different subsets is minimized. Each vertex $v_i \in V$ may be assigned a weight w_i, and each edge $e_{ij} \in E$ may carry a cost c_{ij}. The sum of weights W_k of each partition P_k must not exceed a specified imbalance threshold ϵ relative to the average weight of each partition. An edge can be classified as either an internal edge (those that connect nodes in the same partition), or an external edge (those that connect vertices in different partitions), and the cutsize is defined as the sum of the costs of all external edges. The goal is to minize the cutsize, while preserving the balance between the size of each partition. This problem is known to be NP-Hard, even for obtaining bipartitions on unweighted graphs (adapted from [3]).

Algorithms that aim to solve the graph partitioning problem can only provide approximations, due to the problems inherent difficulty. It is

possible to solve the problem in two trivial ways, each optimizing for only one of the two goals, but both of these methods yield undesriable results. Optimizing for communication minimization can be done by simply assigning all vertices to a single partition, which eliminates all need for communication, but results in extreme load imbalance. Optimizing for load balance can be achieved by randomly assigning vertices to the part of the partition that is currently assigned the smallest number of vertices, but this does not take into consideration the data-dependencies that such an assignment can incur.

3.5.2 METIS

The graph partitioner used for the experiments in this thesis is called METIS. METIS is a software package for partitioning large irregular graphs, and has other uses such as partitioning meshes and computing fill-reducing orderings of sparse matrices. METIS provides algorithms that are based on multilevel graph partitoning. These algorithms partition the graph through a coarsening and refinement phase. In the coarsening phase, the size of the graph is reduced by way of collapsing the vertices and edges, and then partition the smaller graph. In the refinement phase, the graph is gradually expanded to its original size, and the portion of the graph that is close to the boundary is the primary focus, ensuring the quality of the partition is kept (adapted from [8]). Algorithm 3 shows how to partition and reorder the vertices of a graph according to the generated partition, using METIS' K-way partition algorithm.

The way METIS partitions graph does not necessarily optimize for minimizing the communication volume, but rather obtaining partitions that are roughly equal in size. It is possible that there are partitions that have a larger imbalance between the size of partitions, but smaller communication volumes. In order to obtain partitions that optimize for minimizing communication, it is necessary to look to hypergraph partitioners.

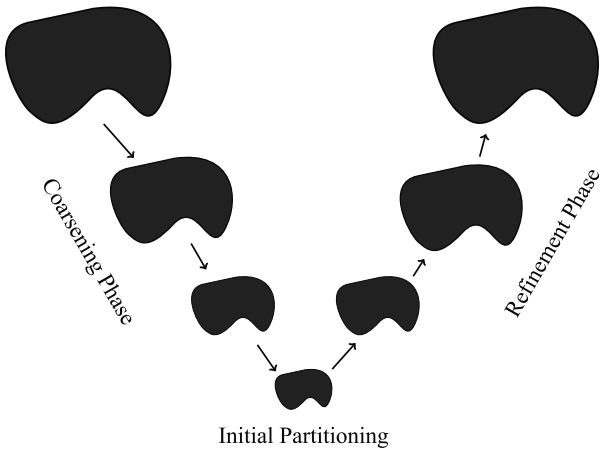


Figure 3.7: Illustration of multilevel graph partitioning (adapted from [8]).

3.5.3 Hypergraph Partitioning

> A *hypergraph* $H = (V, N)$ is defined as a set of vertices V and a set of nets (hyperedges) N among those vertices. Every net $n_j \in N$ is a subset of vertcies, i.e., $n_j \subseteq V$. The vertices in a net n_j are called its *pins* and denoted as $pins[n_j]$. The size of a net is equal to the number of its pins, i.e., $s_j = |pins[n_j]|$. The set of nets connected to a vertx v_i is denoted as $nets[v_i]$. The degree of a vertex is equal to the number of nets it is connected to, i.e., $d_i = |nets[v_i]|$.
>
> Çatalyürek & Ayakanat 1999, p. 7. [3]

Hypergraph partitioners try to solve the same problems, and employ similar algorithms as those graph partitioners use, however they use these algorithms on hypergraphs, and not regular graphs. Çatalyürek and Ayakanat showed in [3] that using the hypergraph partitioner they developed, they could get up to 63% less communication volume, with an average reduction in communication volume of 30% - 38%, when compared to METIS.

As the improvement of the communication strategies are the main focus of the contents of this thesis, the specific partitioner tool that is used is not the biggest concern, and the programs implemented have

opted for the usage of METIS, which as mentioned is a regular graph partitioner. It would be possible to refactor the code in such a way that a hypergraph partitioner such as PaToH can be used.

Algorithm 3: Partitioning and reordering a matrix.

Input : g, n_p, p
Output : Partitioned and reordered g

if $n_p = 1$ **then**
    p[0] ← 0
    p[1] ← $g.n_r$
    return g
partitionVector ← METIS_PartGraphKway(`arguments specifying constraints for partition`)
newId ← $[0] \cdot g.n_r$
oldId ← $[0] \cdot g.n_r$
id ← 0
p[0] ← 0
for $r \in \{0, \ldots r\}$ **do**
    for $i \in \{0, \ldots, g.n_r\}$ **do**
        if *partitionVector[i]* $= r$ **then**
            oldId[id] ← i
            newId[i] ← id
            id ← id + 1
    partitionVector[$r+1$] ← id
newRowPtr ← $[0] \cdot g.n_r + 1$
newColIdx ← $[0] \cdot g.n_c$
newValues ← $[0] \cdot g.n_c$
for $i \in \{0, \ldots, g.n_r - 1\}$ **do**
    d ← g.rowPtr[oldId[i]+1] - g.rowPtr[oldId[i]]
    newRowPtr[i + 1] ← newRowPtr[i] + d
    for $j \in \{0, \ldots, d-1\}$ **do**
        newColIdx[newRowPtr[i] + j] ← g.colIdx[g.rowPtr[oldId[i]] + j]
        newValues[newRowPtr[i] + j] ← g.values[g.rowPtr[oldId[i]] + j]
    for $j \in \{newRowPtr[i], \ldots, newRowPtr[i+1] - 1\}$ **do**
        newColIdx[j] ← newId[newColIdx[j]]
g.rowPtr ← newRowPtr
g.colIdx ← newColIdx
g.values ← newValues
return g

3.6 SuiteSparse Matrix Collection

The matrices that have been used for SpMV in this thesis all come from the SuiteSparse Matrix Collection. This is a large dataset of sparse matrices that come from real world applications. The matrices available in the data set cover problems within structural engineering, CFD, FEM, thermodynamics, optimization, economic and financial modeling, amongst many more. For more information on the SuiteSparse Matrix Collection, see [4].

Chapter 4

Communication Strategies

This chapter introduces the five different communication strategies that have been tested in the experiments conducted for this thesis. Each communication strategy implements increasingly more selective ways of communicating the data-dependencies that arise when the matrix is partitioned amongst the MPI ranks. Starting from the most simple strategy, where the entire input vector x is communicated for each iteration of SpMV, moving on to two separator based methods, and finally two strategies where only the required elements that incur data-dependencies are being communicated.

In the subsequent sections, each figure illustrating a communication strategy is based on the graph shown in Figure 4.1. In this figure each color represents a partition, and the nodes residing within each color belongs to that partition.

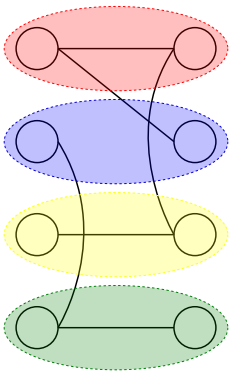


Figure 4.1: Simple graph partitioned amongst ranks - each color represents a rank.

4.1 Strategy A - Exchange entire vector

The first communication strategy presented implements no measures to reduce communication volume. In this strategy, at each iteration every rank sends all of its locally computed values of y to all other ranks so that each rank obtains a complete copy of the output vector before proceeding. This approach can be implemented with the MPI collective operation `MPI_Allgatherv`, which supports different send and receive counts on each rank. Algorithm 4 shows how this communication strategy can be implemented. In practice, using shared memory parallelization instead of this communication strategy would be faster, and easier to implement.

Algorithm 4: Strategy A - Communication Pattern

```
for each iteration do
    spmv(g,x,y)
    MPI_Allgatherv(local_y, sendcount, MPI_DOUBLE, y,
      recvcounts, displs, MPI_DOUBLE, MPI_COMM_WORLD)
    swap pointers of x and y
```

Let n_p denote the total number of ranks, and let $|x_i|$ be the number of vector entries assigned to rank i (given by the average, $\frac{|x|}{n_p}$). Since each rank must send its local block size of $|x_i|$ to the other $n_p - 1$ ranks, the total volume of communication per iteration is given by Equation 4.1.

$$n_p \cdot (n_p - 1) \cdot |x_i| \tag{4.1}$$

Figure 4.2 illustrates the state of the vector before and after communication using this strategy.

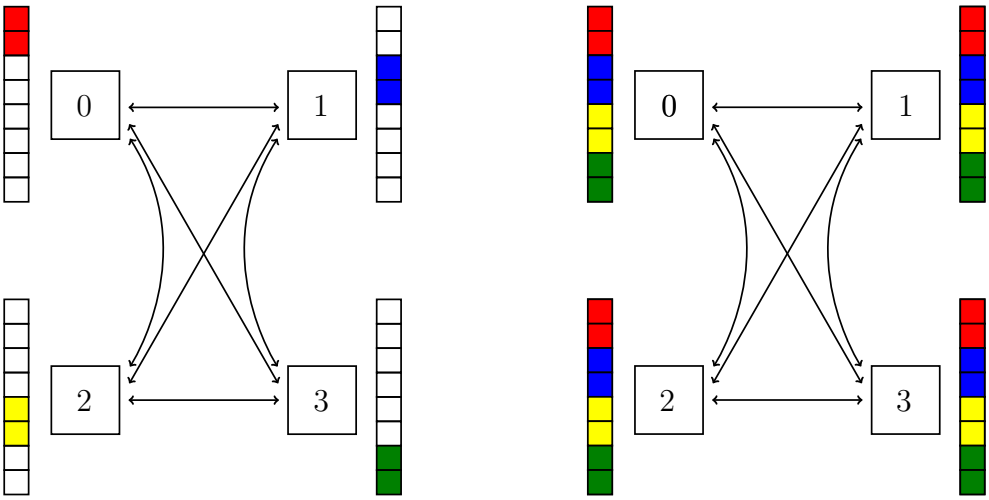


Figure 4.2: Contents of each ranks vector before and after communication under Strategy A.

4.2 Strategy B - Exchange only separators

The next communication strategy improves upon strategy A by recognizing that only the set of each partitions elements of x that induce data-dependencies require explicit communication. The set of these elements is called a separator. This communication strategy is at least as good as Strategy A regardless of the matrix structure. The size of any partitions separator can at most be as large as the set of elements of that partition, but is usually smaller, thus reducing the communication volume. This communication pattern is implemented in the same way as in Strategy A, with the difference being that `sendcount` now stores the size of the separator, instead of the size of the local part of x.

Let n_p denote the total number of ranks, and let s_i be the size of the separator of rank i. Since each rank sends its separator to each other rank (i.e. $n_p - 1$ other ranks), then the total communication volume is given by Equation 4.2.

$$\sum_{i=1}^{n_p} (n_p - 1) \cdot s_i \tag{4.2}$$

To facilitate this strategy, separator values are reordered such that they are stored at the beginning of each ranks local segment of x. Once this structure is established, communication is performed using

`MPI_Allgatherv`, transmitting only the subset of x that contains separator values. The number of separators on each rank must be known beforehand, which can be computed by counting the number of elements that have neighbours belonging to a different partition.

4.2.1 Reordering

After partitioning the matrix into different parts, we obtain a partition vector p, where the $p[i]$ stores the index of the partition the i^{th} entry in `row_ptr`. It is necessary to reorder the entries in `row_ptr` in accordance with the partition vector, such that all entries belonging to the same partition are stored in sequence. The algorithm below gives an outline of how this can be achieved. Here n_p is the number of partitions, n_r is the size of `row_ptr`, and n_c is the size of `col_idx` and `values`.

Algorithm 5: Reordering of Separators

Input : $n_p, n_r, n_c, p, rowPtr, colIdx, values$
Output : Reordered $rowPtr, colIdx, values$

```
newId ← [0] · n_r
oldId ← [0] · n_r
id ← 0
p_0 ← 0

for r ∈ {0, ..., n_p − 1} do
    for i ∈ {0, ..., n_r − 1} do
        if p[i] = r then
            oldId[id] ← i
            newId[i] ← id
            id ← id + 1
    p[r + 1] ← id

newV ← [0] · (n_r + 1)
newE ← [0] · n_c
newA ← [0] · n_c

for i ∈ {0, ..., n_r − 1} do
    degree ← rowPtr[oldId[i] + 1] − rowPtr[oldId[i]]
    newV[i + 1] ← newV[i] + degree

for i ∈ {0, ..., n_r − 1} do
    degree ← rowPtr[oldId[i] + 1] − rowPtr[oldId[i]]
    for j ∈ {0, ..., degree − 1} do
        newE[newV[i]+j] ← colIdx[rowPtr[oldId[i]] + j]
        newA[newV[i]+j] ← values[rowPtr[oldId[i]] + j]
    for j ∈ {newV[i], ..., newV[i + 1] − 1} do
        newE[j] ← newId[newE[j]]

Overwrite rowPtr ← newV, colIdx ← newE, values ← newA
```

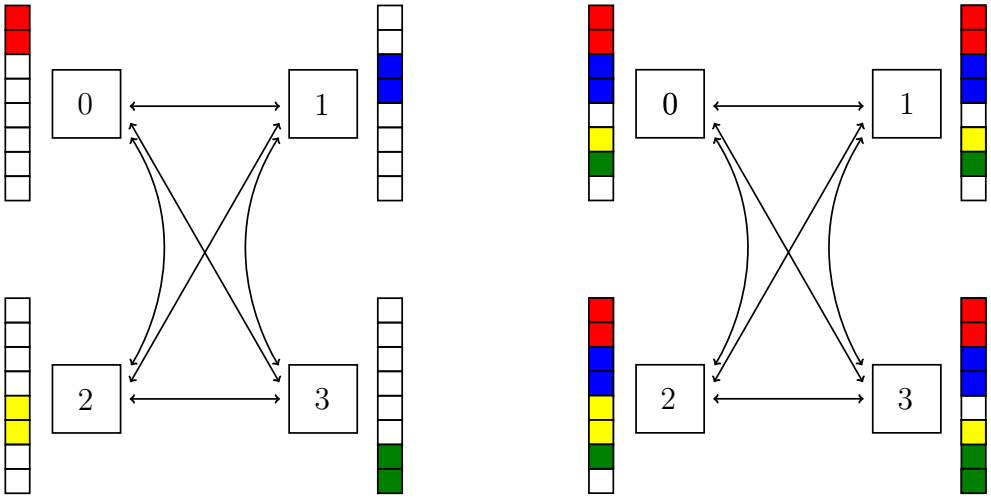


Figure 4.3: Contents of each ranks x vector before and after communication under Strategy B.

4.3 Strategy C - Exchange only required separators

Further reduciton to the communication volume can be achieved by observing that not all separator values are required by every rank. As the number of partitions increases, the set of dependencies between partitions tends towards sparcity. As a consequence of this, certain sets of separators may only need to be communicated to a given subset of the ranks. Using this strategy, each rank only communicates its set of separator values to the ranks that require them. The communication strategy is illustrated in Figure 4.4, and Algorithm 6 shows how this can be implemented.

Let n_p be the total number of ranks, n_i be the number of neighbours of rank i, and s_i the size of the separator of rank i. With $n_i \leq n_p$, the total communication volume is given by 4.3.

$$\sum_{i=1}^{n_p} s_i \cdot n_i \tag{4.3}$$

Algorithm 6: Exchange only required separators

Input : y, rank,size, displacements, sendItems, sendCount
Output :

MPI_Requests ← 0
for *r = 0; r < size; r++* **do**
  if *rank = r or sendItems[r][rank] = 0* **then**
    continue
  Post non-blocking MPI receive for $sendCount[r]$ elements at address $y + displacements[r]$
  MPI_Requests++
for *r = 0; r < size; r++* **do**
  if *rank = r or sendItems[r][rank] = 0* **then**
    continue
  Post non-blocking MPI send of $sendCount[rank]$ elements to address $y + displacements[rank]$
  MPI_Requests++
Wait for all non-blocking requests to complete

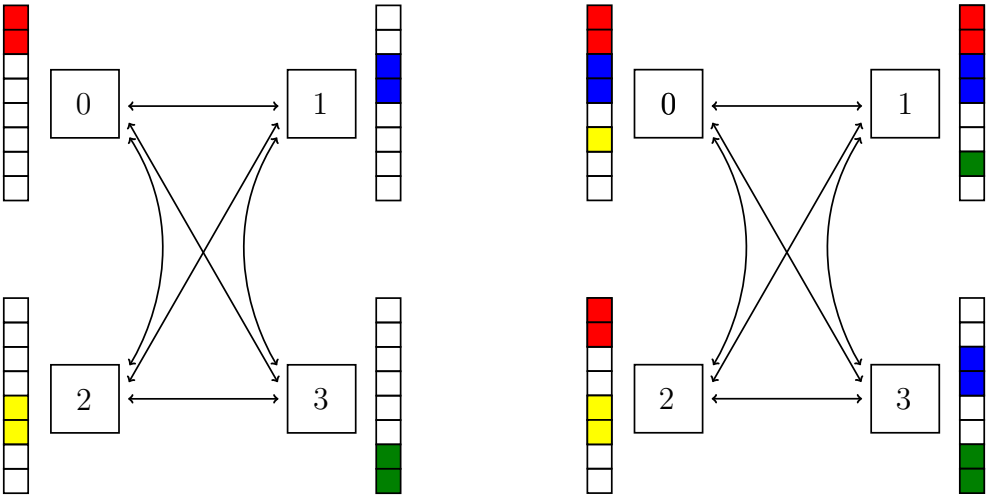


Figure 4.4: Contents of each ranks x vector before and after communication under Strategy C.

4.4 Strategy D - Exchange only required separator values

The final strategy further reduces the communication volume by only communicating the separator elements to the rank with which that element induces a data dependency. This eliminates unnecessary communicaton, meaning that this strategy is communication minimal, but introduces complexity in managing said communication. It requires careful management of the send and recieve lists, and introduces some additional overhead by means of a packing and unpacking step necessary for communicating the separator elements between ranks.

Algorithm 7 shows how the communication step for this strategy is implemented. Both Strategy B and C requires reordering of the separator values, in order to make the communication of the separators easier. In contrast, Strategy D does not require reorderding of the separators, as it packs each of the separators into send lists before communication. The original index of the separator element is stored, and is used to unpack it into the correct position of x after communication.

Let n_p be the total number of ranks, and let d_i be the number of elements in rank is separator that are required by other ranks. Since this communication strategy only communicated these elements, the total communication volume is given by 4.4.

$$\sum_{i=1}^{n_p} d_i \tag{4.4}$$

Algorithm 7: Strategy D - Packing, exchanging and unpacking separator elements

```
Input  : c, x, rank, size
Output : Updated vector x
totalSend, totalRecv ← 0
for i = 0; i < size; i++ do
    totalSend ← totalSend + c.sendCount[i]
    totalRecv ← totalRecv + c.receiveCount[i]
Allocate sendBuffer[totalSend], recvBuffer[totalRecv]
sendOffset ← 0
for i = 0; i < size; i++ do
    for j = 0; j < c.sendCount[i]; j++ do
        sendBuffer[sendOffset++] ← x[c.sendItems[i][j]]
Compute sendDispls, recvDispls as prefix sums of c.sendCount,
 c.receiveCount
MPI_Ialltoallv(sendBuffer, c.sendCount, sendDispls, recvBuffer,
 c.receiveCount, recvDispls)
MPI_Waitall()
recvOffset ← 0
for i = 0; i < size; i++ do
    for j = 0; j < c.receiveCount[i]; j++ do
        x[c.receiveItems[i][j]] ← recvBuffer[recvOffset++]
Free sendBuffer, recvBuffer, sendDispls, recvDispls
```

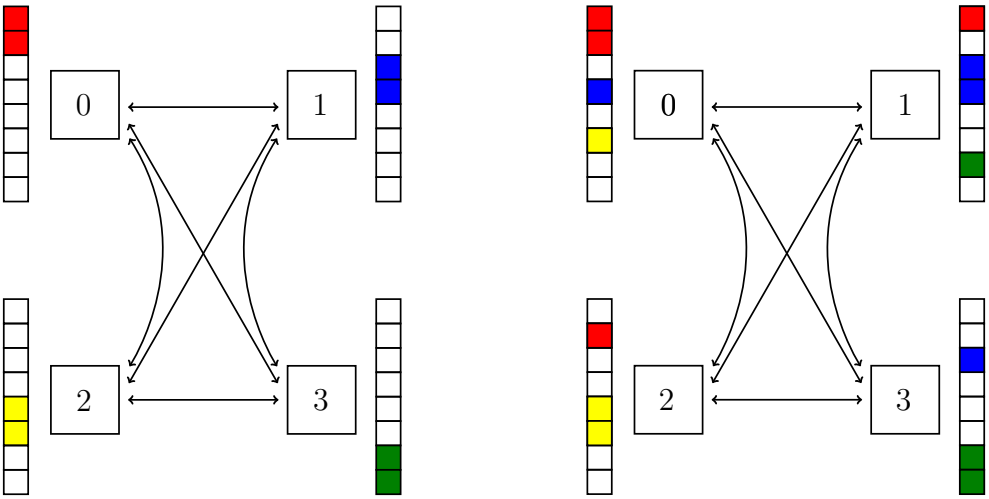


Figure 4.5: Contents of each ranks x vector before and after communication under Strategy D.

4.5 Strategy E - Memory Scalable

The communication strategies discussed so far all have a common problem that prevents them from scaling to large matrices. These strategies all store the entire x vector, and as matrices get larger, so the x vector. As the chips have a limited amount of memory, they will eventually run out of memory when the vector gets large enough. When this happens, the vector has to be read from disc instead of from memory, which in turn will essentially kill all performance gained from moving to a parallel environment. This strategy provides a solution to that, by implementing measure to ensure memory scalability.

Instead of storing the entire vector, each rank only stores the elements of x that reside within that ranks partition. In addition to storing the local part of x, the separator elements are also stored consecutively in memory after the local elements of x. Doing this requires reodering of the local x vector, aswell as the separator elements. Figure 4.6 illustrates the contents of the x vector before and after communication. The communication volume is exactly the same as for Strategy D, but as is seen this time only the necessary elements are stored in memory, which means that this communication strategy is both communication minimal, but also memory minimal.

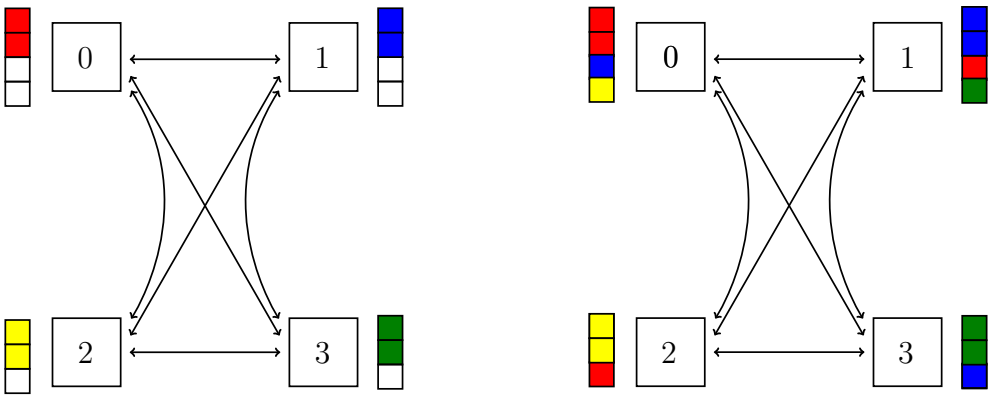


Figure 4.6: Contents of each ranks x vector before and after communication under Strategy E.

Even though the communication pattern in Strategy D and E is the same, their exact implementation differs. Strategy D relies on `MPI_Ialltoallv` as is seen in Algorithm 7, but Strategy E uses a custom implementation of all to all communication, that relies on `MPI_Send` and `MPI_Irecv` internally. This does have some impact on performance, as will be seen in Section 5.4

Table 4.1 gives an overview of the reduction in of the total communication volume for the strategies described in the previous sections. As the graph in Figure 4.1 is very small, this table doesn't illustrate the full potential of any of the strategies, but it does show that the communication volume is reduced for each strategy.

Table 4.1: Overview of total communication volume reduction for strategy A-E using graph in Figure 4.1.

Strategy	**% of x communicated**	**% diff. from prev. strategy**
A	100	N/A
B	75	25
C	41.7	33.3
D	25	16.7
E	25	0

Chapter 5

Results

5.1 Theoretical Maximum

As has been established, the maximum performance scales with the available memory bandwidth of the system, and not with the amount of processes used. For SpMV, a total of 6 bytes are read per FLOP, which means that the theoretical maximum is given by Equation 5.1. This is under the assumption that the entire memory bandwidth is available during SpMV, which is not the case in reality (see [18]).

$$\text{Maximum Theoretical Performance} = \frac{\text{Memory bandwidth}}{6} \tag{5.1}$$

5.2 Experimental Setup

In order to rigorously evaluate and compare the performance of the five communication strategies, a systematic series of experiments were ran on the eX^3 machine. Each experiment executes 100 iterations of SpMV on a given sparse matrix, with a given configuration and a given communication strategy. Every program used has been compiled GCC 11.4.0 using the compiler flags `-march=native` and `-O3`. The code used to produce the results for the communication strategies is publically available, see the Appendix.

5.2.1 Hardware Platforms

The experiments that have been ran use three different high-performance computing systems. Table 5.1 gives an overview of the hardwares and their specifications that has been used in the experiments.

Table 5.1: Hardware used in the experiments. STREAM Triad was run with the `-march=native` and `-O3` compilation flags.

	FPGAQ	Rome	Defq
CPUs	AMD EPYC 7413	AMD EPYC 7302P	AMD EPYC 7601
Instr. setu w	x86-64	x86-64	x86-64
Microarch.	Zen 3	Zen 2	Zen 1
Sockets	2	1	2
Cores	2×24	1×16	2×32
Freq. [GHz]	2.6–3.6	1.5–3.3	2.2–3.2
L1I/core [KiB]	64	32	64
L1D/core [KiB]	32	32	32
L2/core [KiB]	512	512	512
L3/socket [MiB]	128	16	128
Mem. channels	2×8	1×8	2×8
Bandwidth [GB/s]	N/A	204.8	342
Triad [GB/s]	409.6	90.9	169.7

There has been performed experiments measuring both single-node and multi-node performance. For single-node experiments, the number of MPI ranks is doubled until all cores on the given node is assigned a MPI rank, also OpenMP was not utilized for these experiments.

For multi-node experiments, two configuration have been tested. The first configuration involves assinging one MPI rank per node, and utilizing shared memory paralleization through the use of OpenMP within each node. The second configuration places one MPI rank per socket. These experiments aims to show the performance of the communication strategies on multi-node systems. Furthermore, it is possible to compare the impact that the number of memory controllers utilized have on the results. Using one MPI rank per node only utilizes one of two memory controllers on a dual socketed systems, whereas using one MPI rank per socket utilizes both of the memory controller on the node.

5.2.2 Communication Strategies

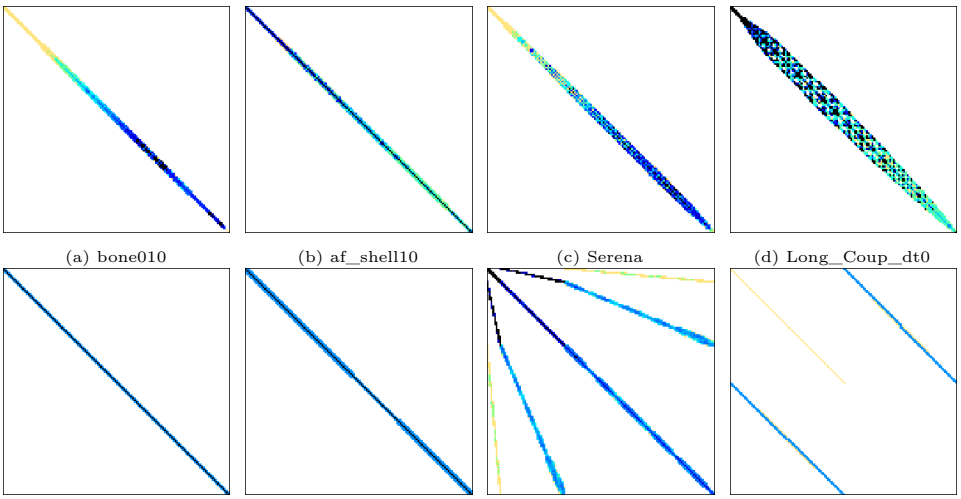
The communication strategies used have been extensively discussed in 4, and Table 5.2 gives an overview of what the various communication strategies will be referred to in the folliwing sections, and the key idea they use to communicate the data-dependencies between ranks.

Table 5.2: Names and description of communication strategies that have been tested.

Strategy name	**Strategy Description**
Strategy A	Exchange entire local vector.
Strategy B	Exchange entire separator.
Strategy C	Exchange required separator.
Strategy D	Exchange required separator elements.
Strategy E	Exchange required separator elements, memory scalable.

5.2.3 Matrices

As the field of interest is that of results from scientific computing, the matrices shown in Figure 5.1 have been selected, and Table 5.3 provides specifications for the matrices. Matrices that are based on scientific computing problems are generally well structured and have nonzero entries centered around the main diagonal. Matrices with these kinds of structures tend to yield a larger cache hit rate than matrices based on social networks and other kinds of graphs (see [18]). It is important to keep this in mind when interpreting the results, as other kinds of graphs can give very different results.



(a) bone010 (b) af_shell10 (c) Serena (d) Long_Coup_dt0

(e) dielFilterV3real (f) Cube_Coup_dt0 (g) Bump_2911 (h) nlpkkt200

Figure 5.1: Matrices used in the following experiments. (Matrix pattern images collected from the University of Florida Sparse Matrix Collection [4], accessed via the SuiteSparse Matrix Collection Website Interface [9].)

Table 5.3: Specifications of the matrices used in experiments. (Matrix data collected from the University of Florida Sparse Matrix Collection [4], accessed via the SuiteSparse Matrix Collection Website Interface [9].)

Matrix	Num Rows	Num Cols	Nonzeros	Symmetric
Bump_2911	2 911 419	2 911 419	127 729 899	Yes
Cube_Coup_dt0	2 164 760	2 164 760	124 406 070	Yes
Long_Coup_dt0	1 470 152	1 470 152	84 422 970	Yes
Serena	1 391 349	1 391 349	64 161 971	Yes
af_shell10	1 508 065	1 508 065	52 259 885	Yes
bone010	986 703	986 703	47 851 783	Yes
dielFilterV3real	1 102 824	1 102 824	89 306 020	Yes
nlpkkt200	16 240 000	16 240 000	440 225 632	Yes

5.3 AMD EPYC 7601

For experiments on the dual-socketed AMD EPYC 7601 nodes, three sets of experiments were conducted. The first set of experiments test the performance when using a single-node without shared memory paralleization, The second set places only one MPI rank per node, and the final set of experiments place one MPI rank per socket. Both multi-node experiments use shared memory parallelization for intra node communication.

5.3.1 Single-node Performance

For the single-node experiments, no shared memory paralleization has been utilized. Each communication strategy has been ran using $1, 2, 4, 8, 16, 32$ and 64 MPI ranks, as the dual socketed system provides two 32-core AMD EPYC 7601 chips.

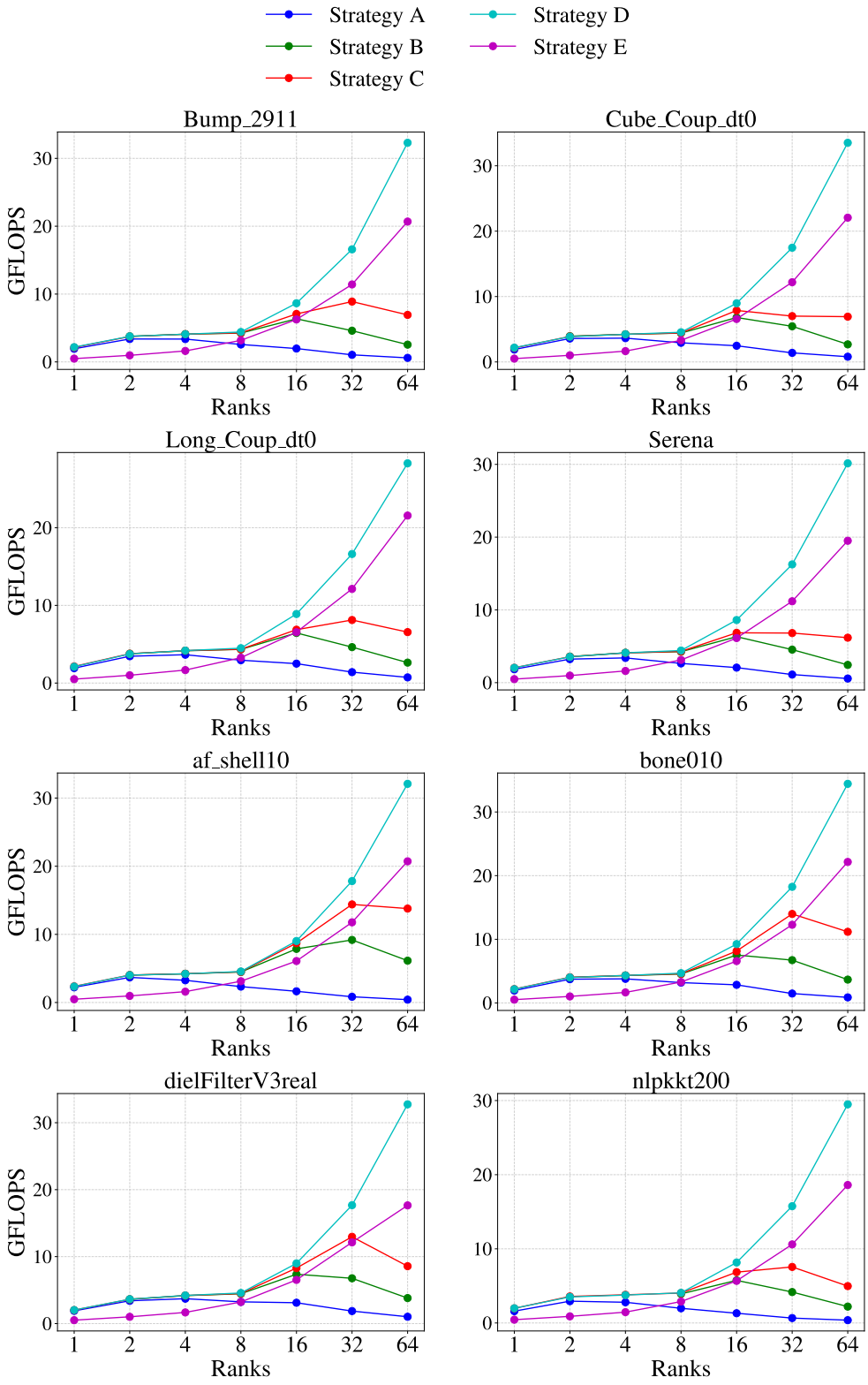


Figure 5.2: Sustained performance (GFLOPS) over 100 iterations of SpMV on a single node equipped with dual-socket AMD EPYC 7601 processors.

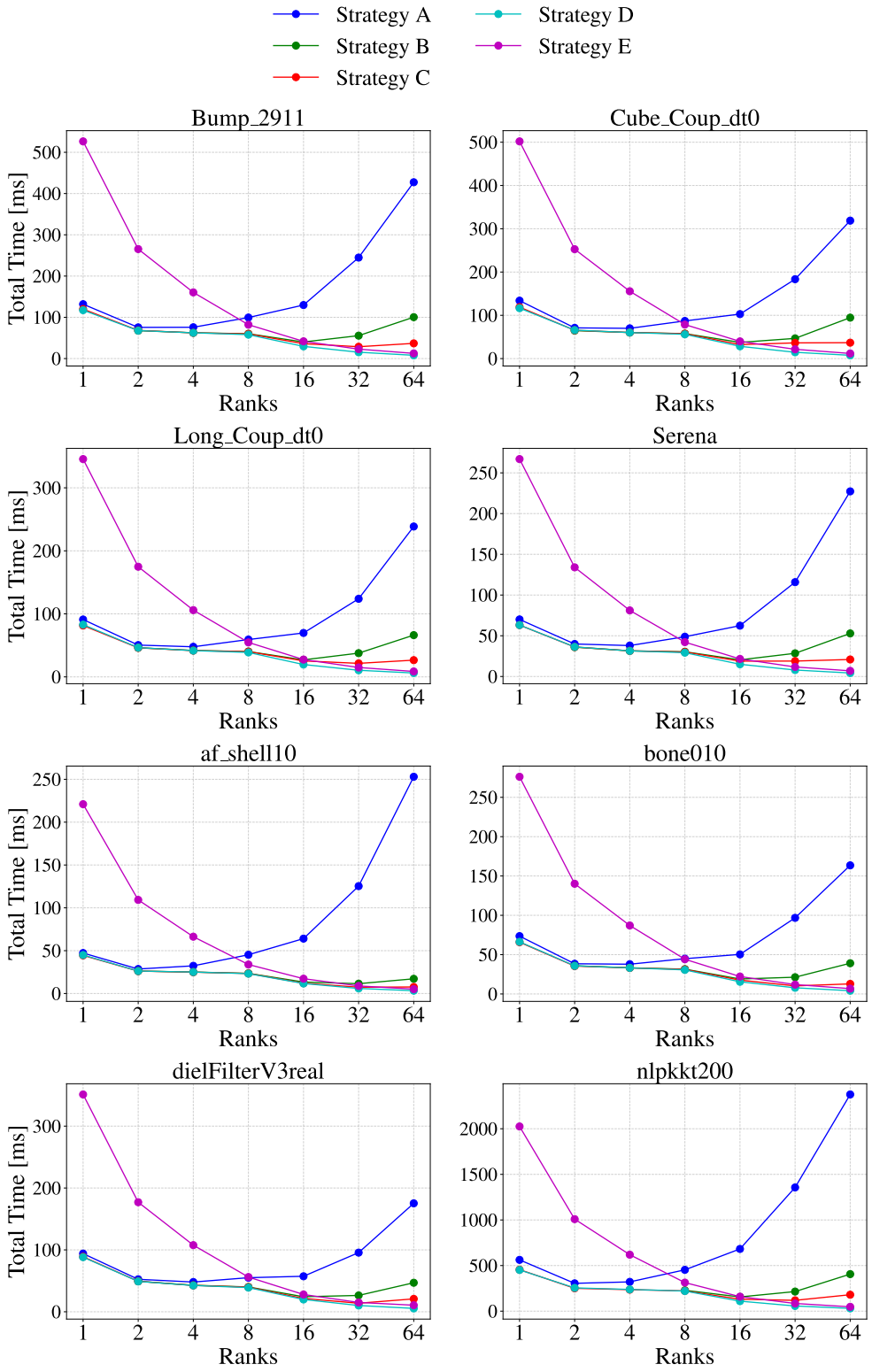


Figure 5.3: Total execution time per iteration of SpMV on a single node equipped with dual-socket AMD EPYC 7601 processors.

Figure 5.2 presents the achieved GFLOPS for each communication strategy. The performance trends largely align with expectations, where each successive strategy exhibits improved performance over its predecessor. However, an exception arises with Strategy E, which consistently underperforms relative to Strategy D.

As discussed in Section 4.5, Strategy E reorders the local vector and separators. The effect this reordering has on well ordered matrices is that it can reduce cache reuse, as it can make the structure of the matrices worse. This has not been measured directly, but it is the most likely culprit to the discrepancy between the results of Strategy D and E. Trotter et. al conducted a study on the reordering of matrices, and they also found that the reordering of the matrix worsened the performance on AMD EPYC 7601 chips [18].

Figure 5.3 presents the total execution time for each communication strategy. It is closely related to Figure 5.2, as computing the GFLOPS number is given by the total number of flops performed divided by the total execution time.

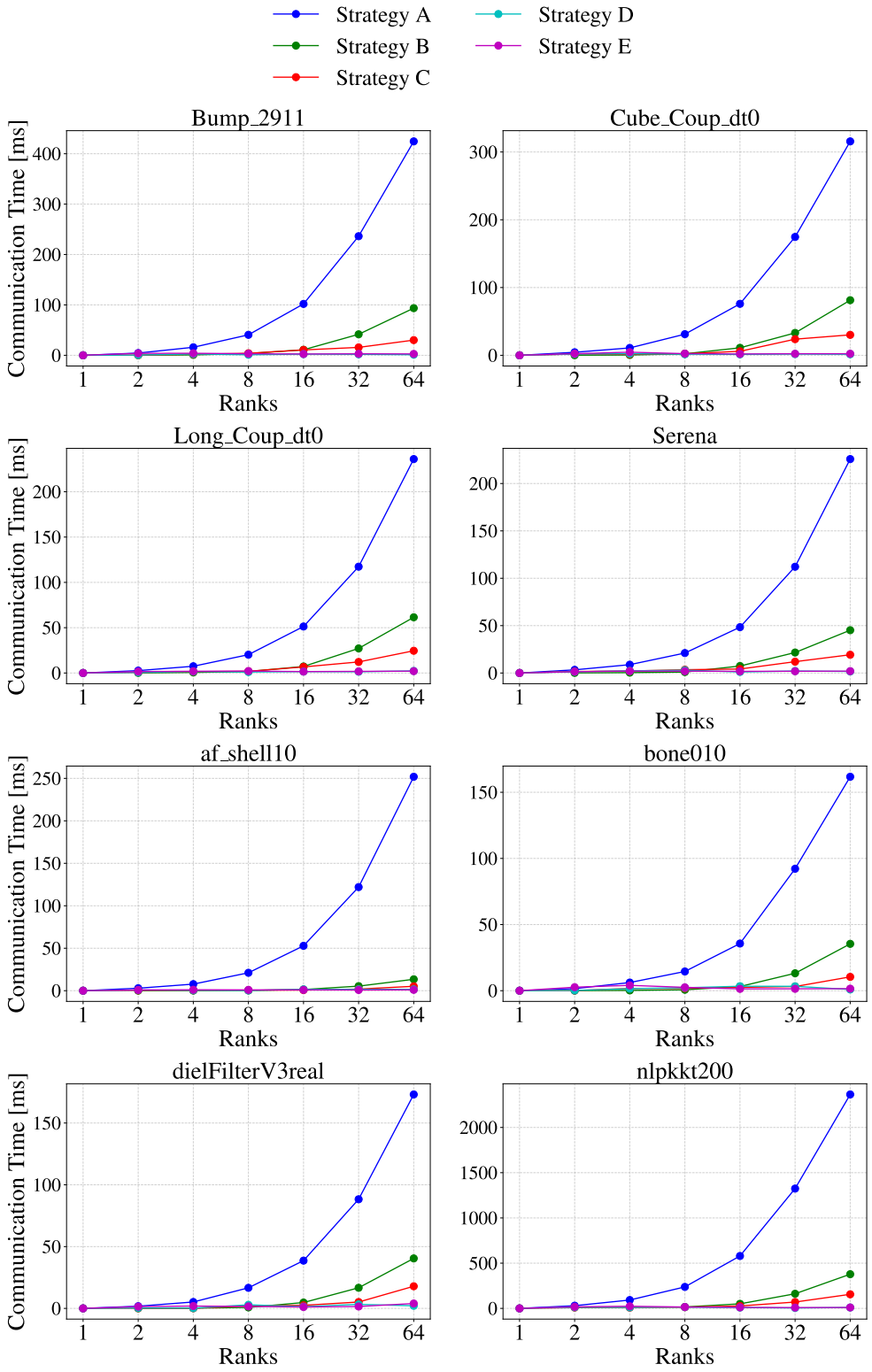


Figure 5.4: Communication time per iteration of SpMV on a single node equipped with dual-socket AMD EPYC 7601 processors.

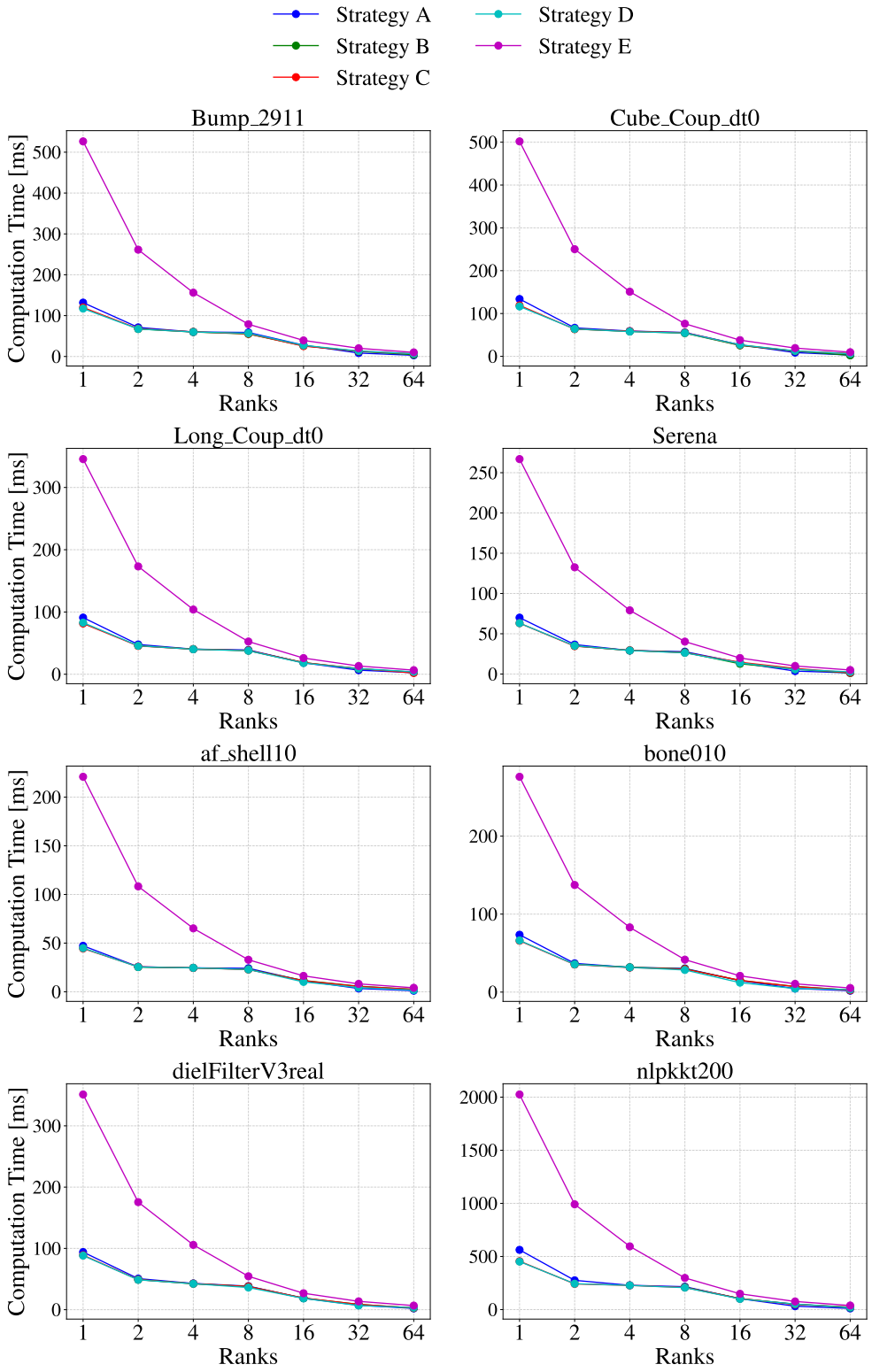


Figure 5.5: Computation time per iteration of SpMV on a single node equipped with dual-socket AMD EPYC 7601 processors.

Figure 5.5 and 5.4 present the computation time and communication time, respectively, of each communication strategy. From these figures, it becomes evident that as the number of ranks is increased, all strategies go from being compute-bound (i.e. the computation time dominates the execution time), to being communication-bound (i.e. the communication time dominates the execution time).

Strategy A performs the worst, which is expected. Strategy B results in a 3-10x reduction in communication time compared to Strategy A when comparing the results of using 64 MPI ranks. Further reductions are observed in Strategy C, with Strategy D and E performing the best.

Due to the reordering, Strategy E starts off as the worst performing strategy, but catches up in performance when scaling to 16, 32 and 64 MPI ranks. As is evident in Figure 5.5, Strategy A-D performs comparably, as these strategies don't reorder the matrix, and have essentially the same memory access patterns.

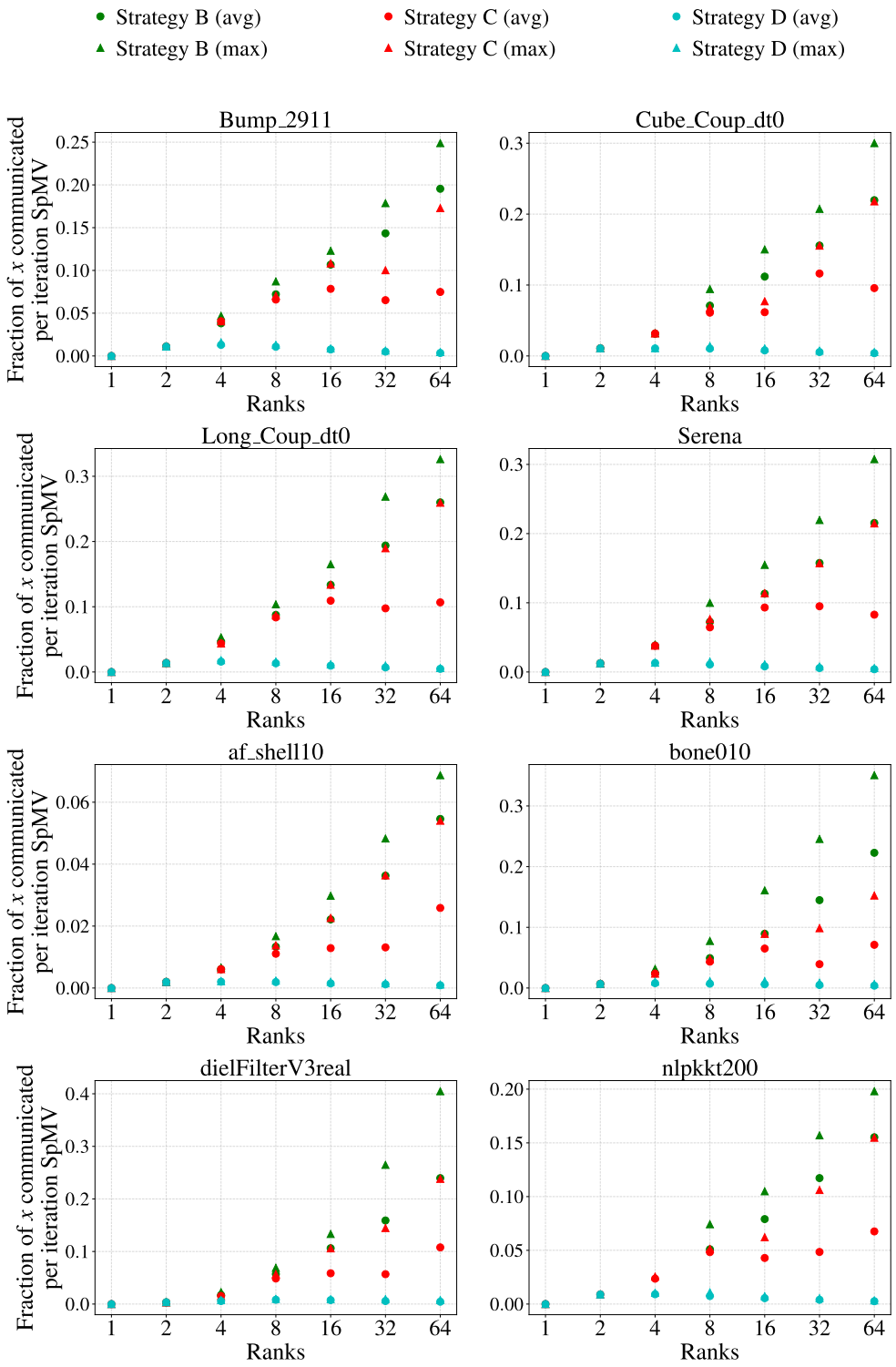


Figure 5.6: Fraction of the size of the global x vector communicated per SpMV iteration.

Figure 5.6 illustrates the fraction of the global size of x that is communicated for each iteration of SpMV. This provides important insight into the reason for the differences between the communication times of the strategies.

Using one rank, no communication occurs, and with two ranks, Strategy B-D performs the same amount of communication as the entire separator from one rank is required from the other, and vice-versa.

From this figure, observe that both the average and maximum communication volume is reduced in each communication strategy. Note how the maximum communication of both Strategy B and C is considerably higher than the maximum of Strategy D. This is interesting as the communication routines used to facilitate communication between ranks is blocking, meaning that the communication time is partially dictated by the time the rank with the maximum communication volume spends sending its messages.

5.3.2 Multi-node Performance

The multi-node experiments illustrate the performance across 1-4 dual-socketed nodes. Two configurations have been tested, these being using one MPI rank per node with 64 OpenMP threads per rank, which utilizes only one of the two memory controller per node. Next, Two MPI ranks per node, or one MPI rank per socket, with 32 OpenMP threads per rank, which utilizes both memory controllers available per node.

One MPI rank per node

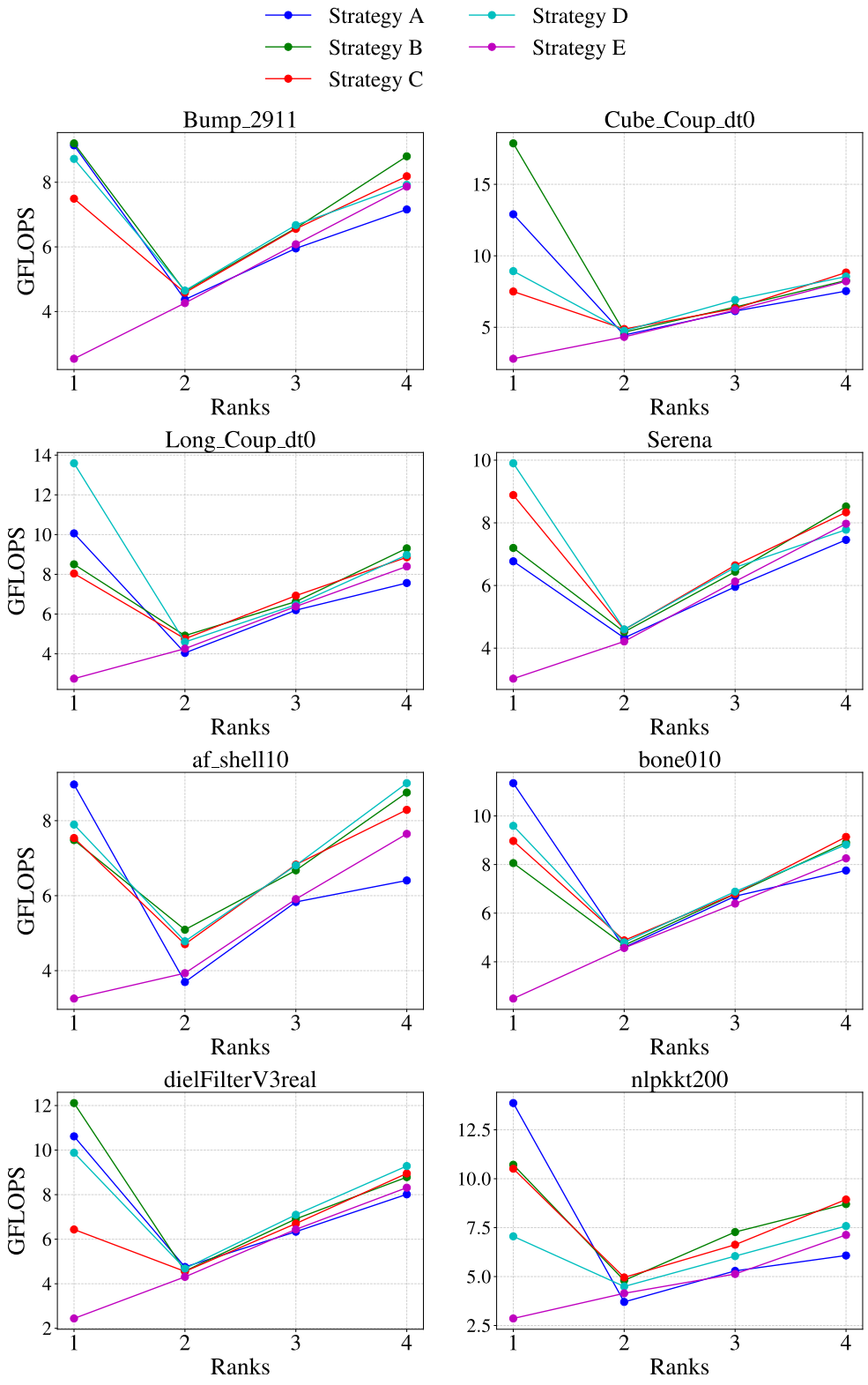


Figure 5.7: Sustained GFLOPS performance of SpMV over 100 iterations on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

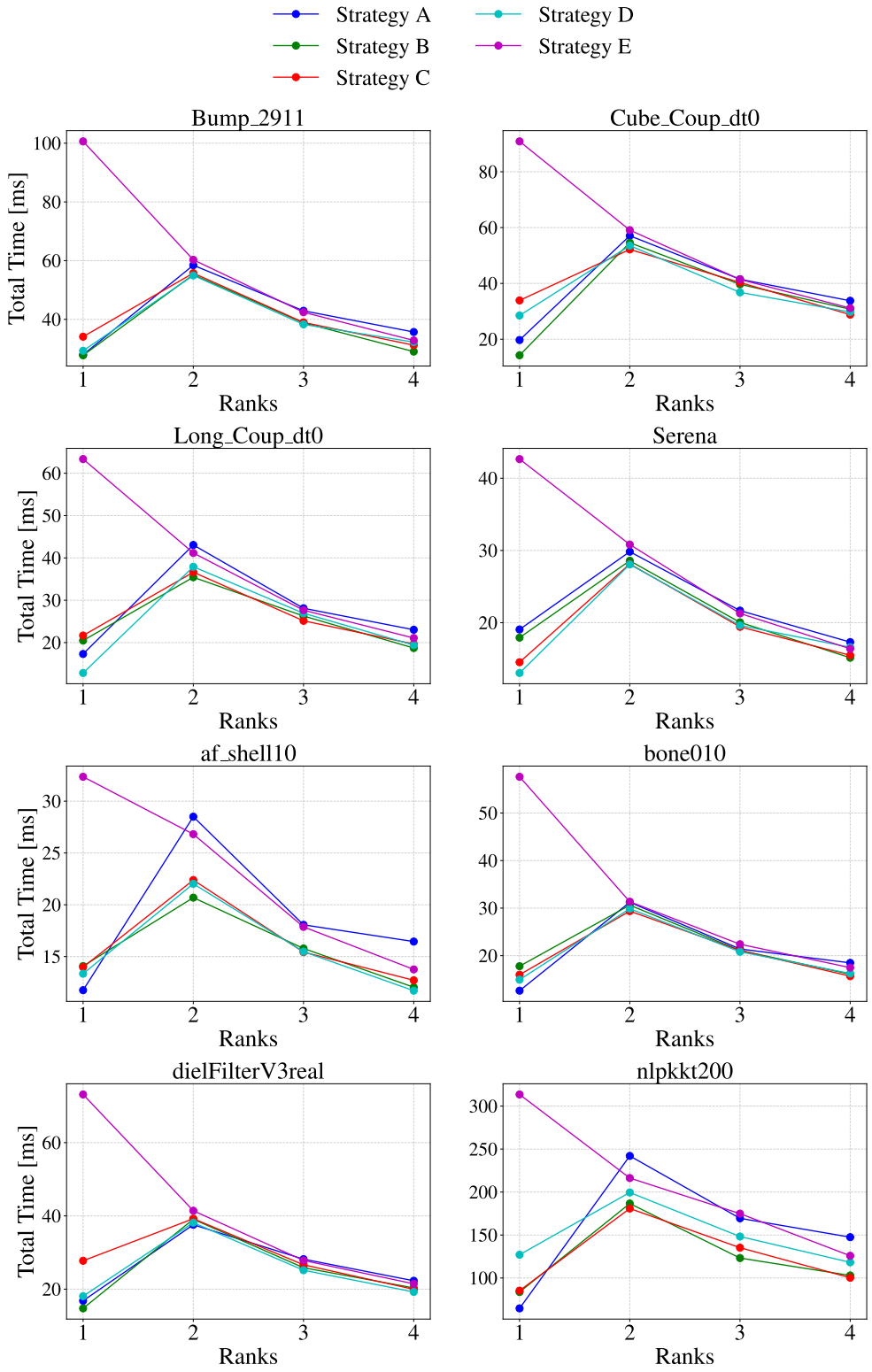


Figure 5.8: Total execution time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

The results in Figure 5.7 and 5.8 illustrate the sustiained GFLOPS performance, and total executution time per iteration SpMV. This configuration uses one MPI rank per node. As is evident from the plots, using only one node (and one MPI rank), consistently yield results that are either better or almost as good as the results achieved when using the maximum amount of available nodes, and when compared to using 64 MPI ranks, one per physical core, the performance is much worse. As an example, for the `Bump_2911` matrix, Strategy D achieves a GFLOPS performance of ≈ 9 GFLOPS on one node, with 64 OpenMP shared memory threads. When this is compared to Figure 5.2 using 64 MPI ranks, which achieves ≈ 30 GFLOPS for the same matrix.

From these results, it is clear that as soon as the network cards connecting the nodes are accessed, performance sharply drops. The exact reason for this is outside the scope of this thesis. Thune et. al [17] performed a study investigating point-to-point MPI communication, and their findings yielded similar degredations in performance when moving to inter-node communications.

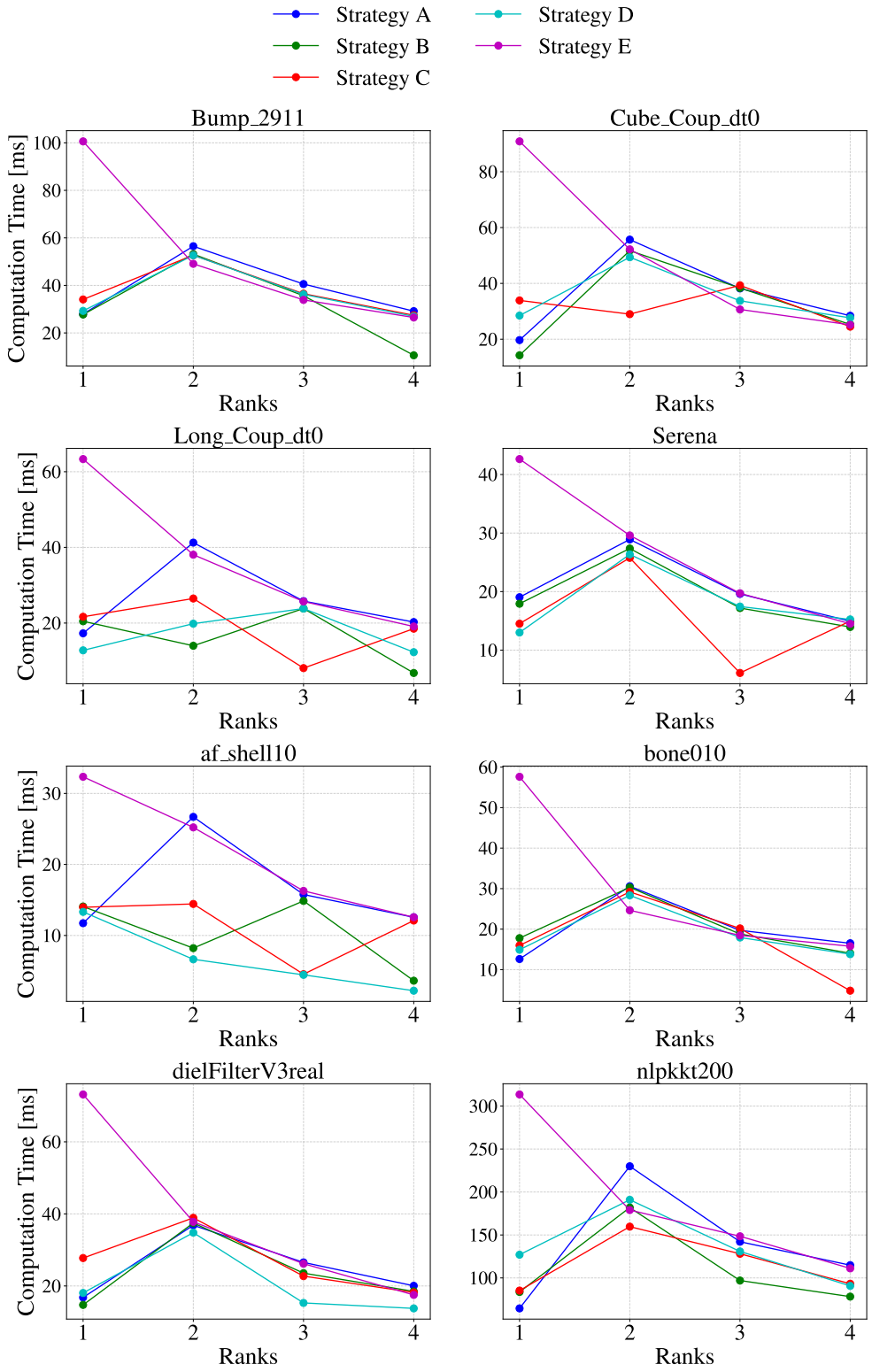


Figure 5.9: Computation time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

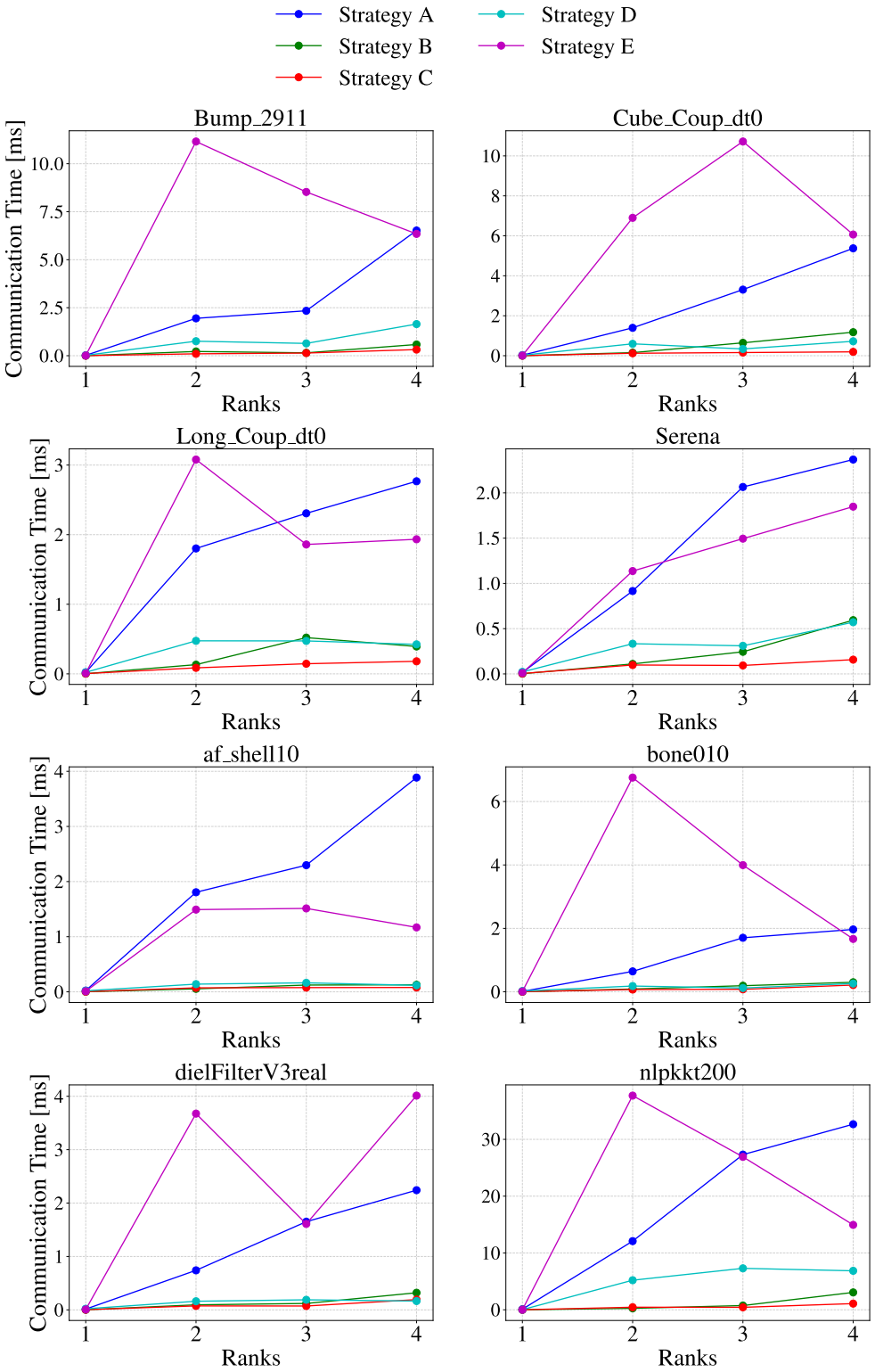


Figure 5.10: Communication time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

Figure 5.9 and 5.10 illustrates the computation and communication time respetively per iteration of SpMV. For the first plot, nothing out of the ordinary is observed, except for some erraticism on some of the matrices. For the second plot however, we can see that the communication time of Strategy E strats of significantly higher than the other strategies, before sharply dropping as the number of ranks increases. We can observe the start of some trends, with both Strategy B and C increasing in communication time, Strategy D remaining relatively stable, and Strategy E approaching that of Strategy D. The reasons for worse performance of Strategy E will be discussed in Section 5.4.

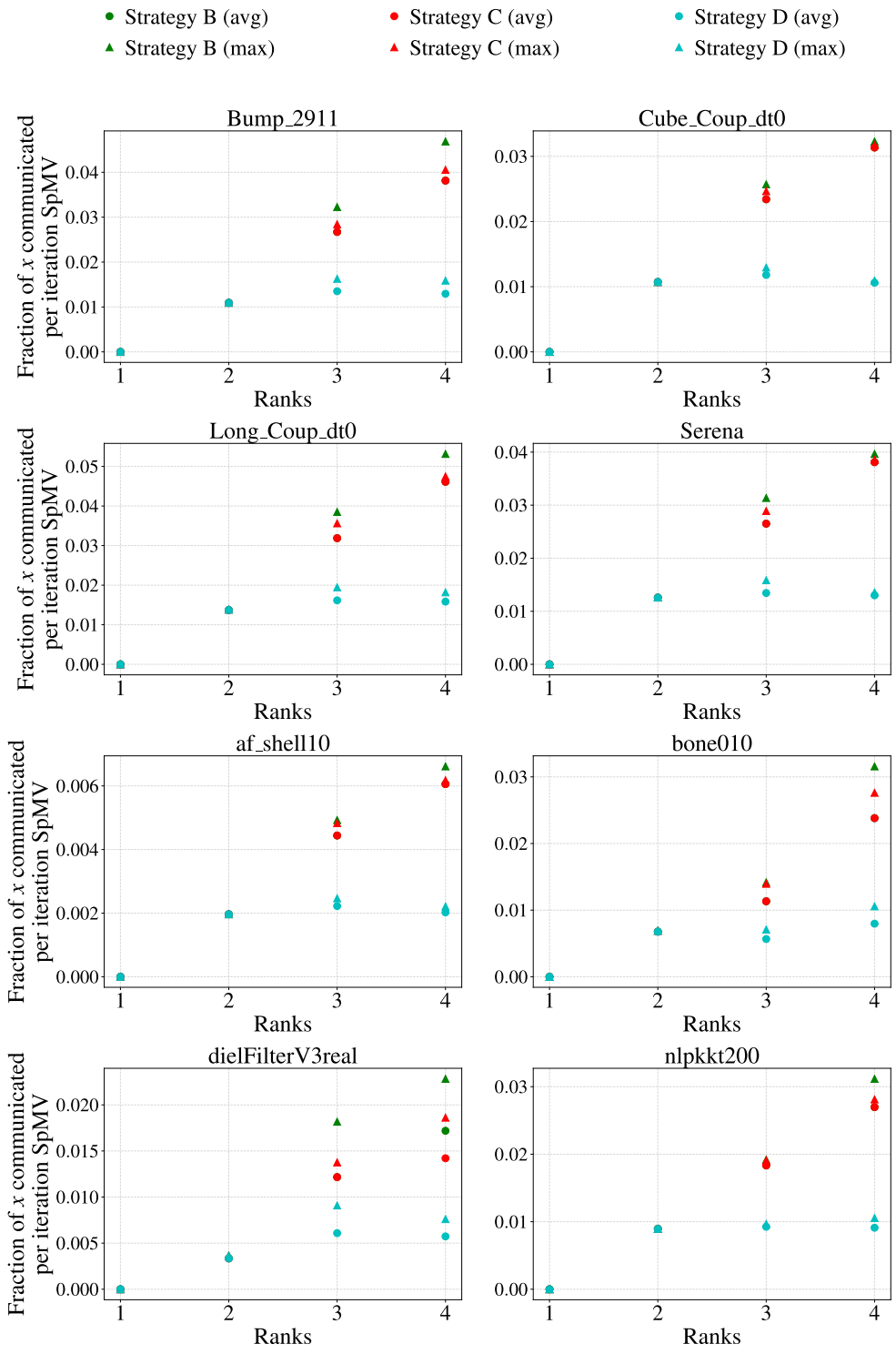


Figure 5.11: Fraction of the global x vector communicated per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7601 processors.

One MPI rank per socket

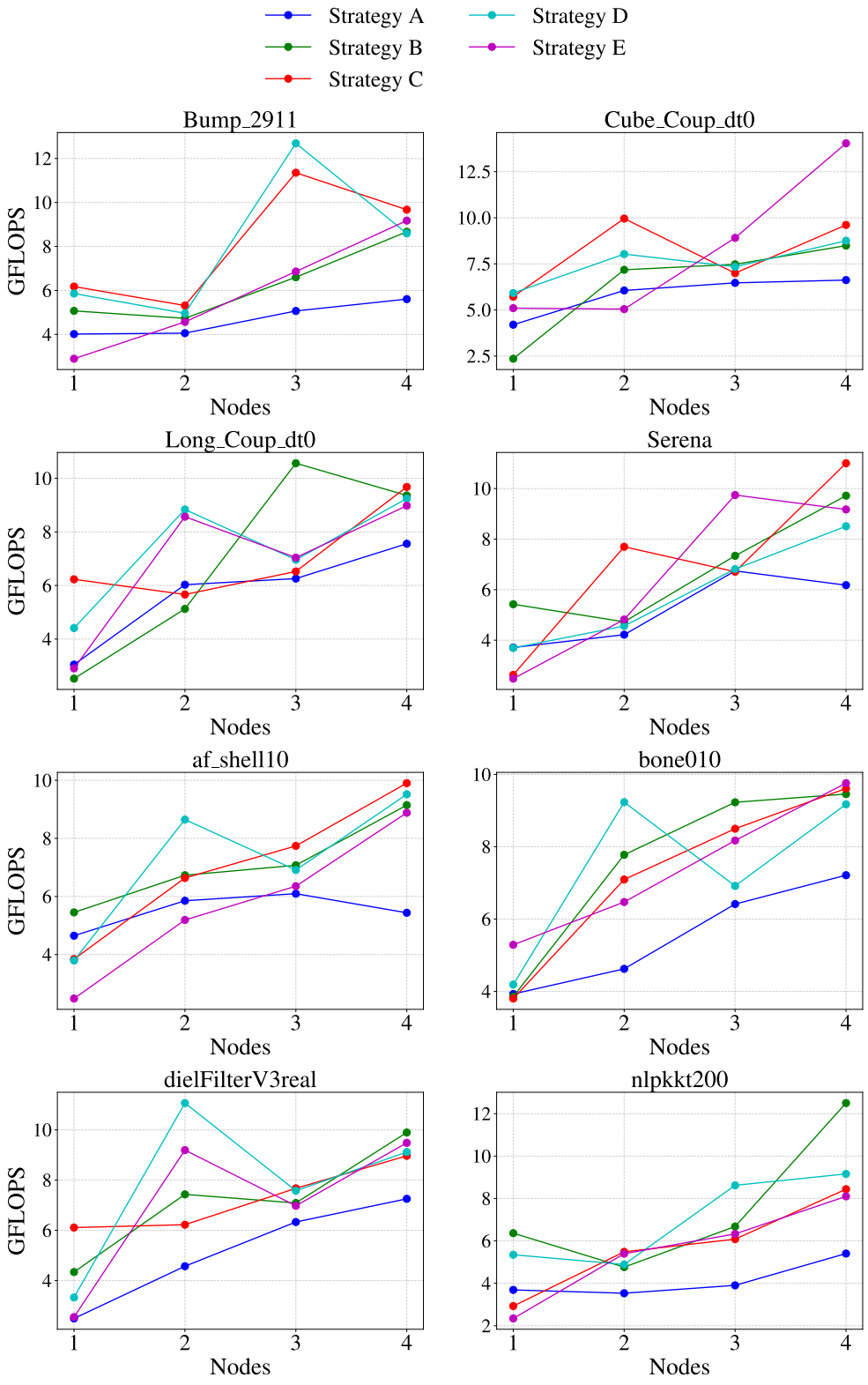


Figure 5.12: Sustained GFLOPS performance of SpMV over 100 iterations on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7601 processors.

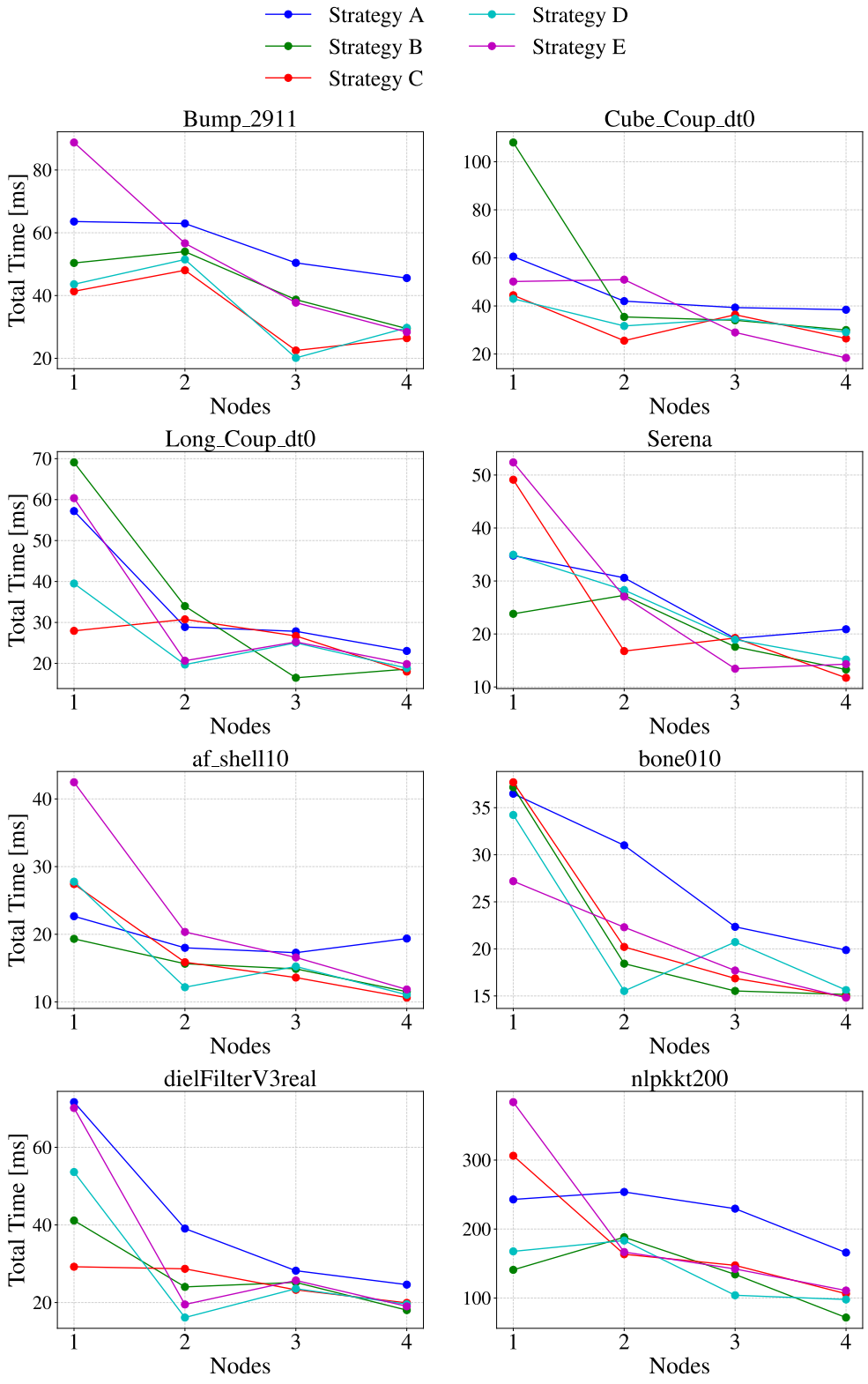


Figure 5.13: Total execution time per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7601 processors.

Figure 5.12 and 5.13 illustrate the sustained GFLOPS performance achieved across 100 iterations of SpMV, and the execution time per iteration of Spmv, respectively. When compared to the results in Figure 5.7 and 5.8, we see that the performance is better in the multi node case by $\approx$ 1-2 GFLOPS in most matrices. We also observe a greater spread in performance across the strategies for all configurations.

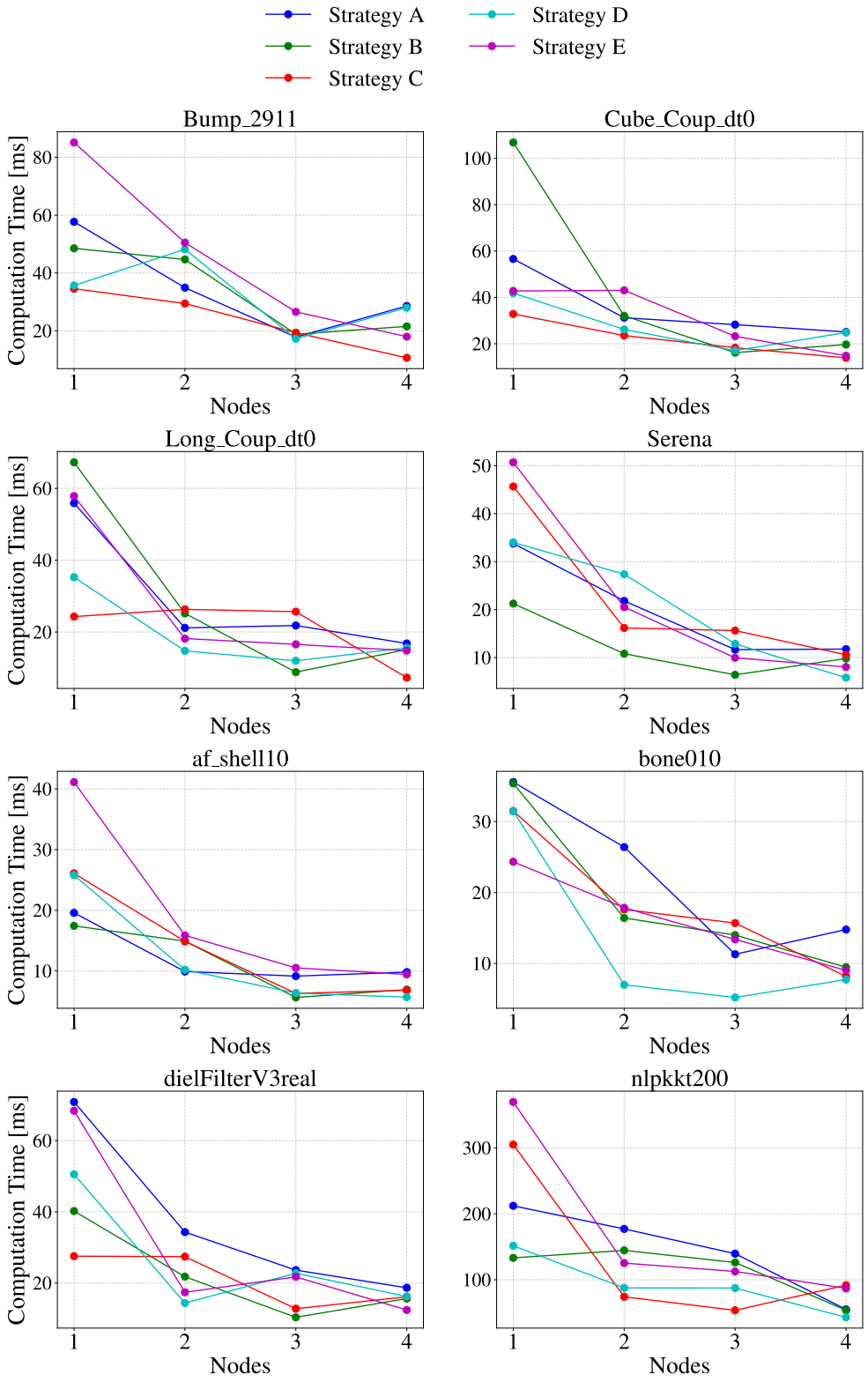


Figure 5.14: Computation time per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7601 processors.

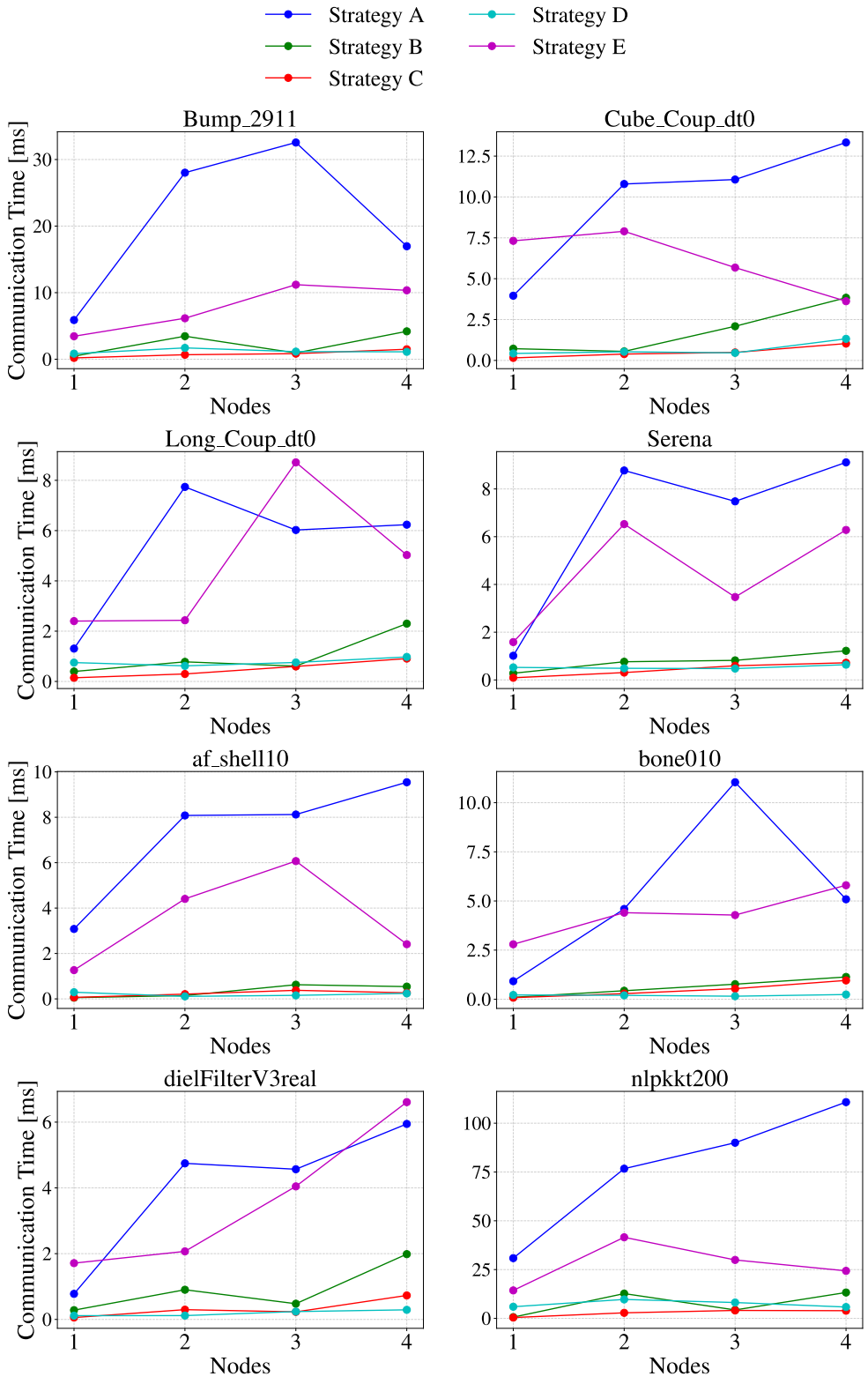


Figure 5.15: Communication time per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7601 processors.

Figure 5.14 and 5.15 illustrate the computation time and communication time for each iteration of SpMV, respectively. We can observe that the computation time decreases as the number of nodes increases, which is expected. As was the case for Figure 5.10 Strategy E performs worse, but does exhibit trends in decreasing the communication time as the number of nodes increases. Notably, Strategy C (and Strategy B on some matrices) yield communication times comparable to that of Strategy D, with Strategy C beating D in some matrices. This indicates that in the multi-node setting, communication minimal strategies don't always results in the lowest communication times.

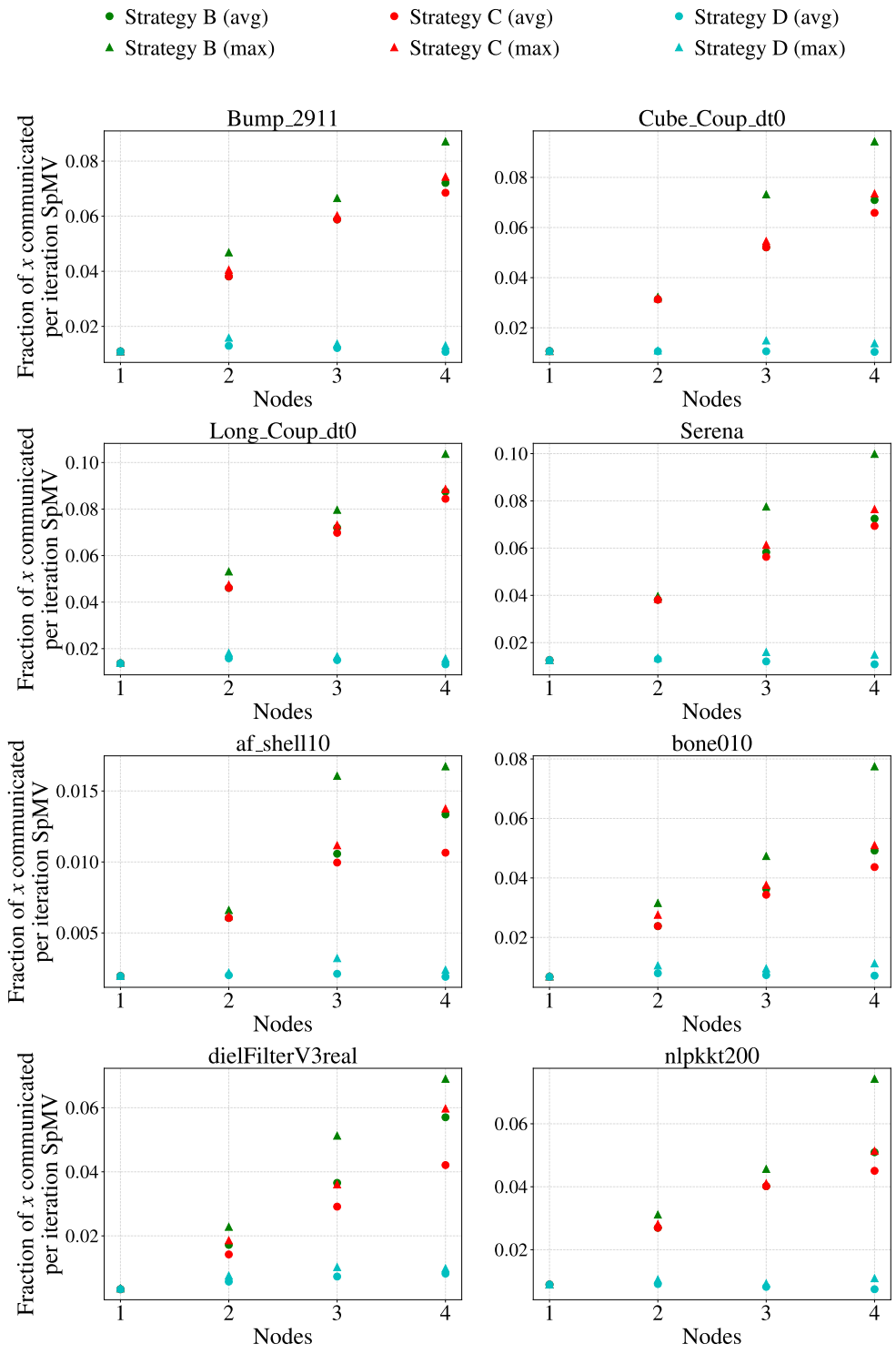


Figure 5.16: Fraction of the global x vector communicated per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7601 processors.

5.4 AMD EPYC 7302P

The results in this section present the performance metrics of experiments ran on 1-8 nodes with a single-socketed AMD EPYC 7302P chip. As the nodes are single socketed, only one set of experiments are provided, where one MPI rank is placed per node, each with 16 OpenMP threads, as the AMD EPYC 7302P chip has 16 physical cores.

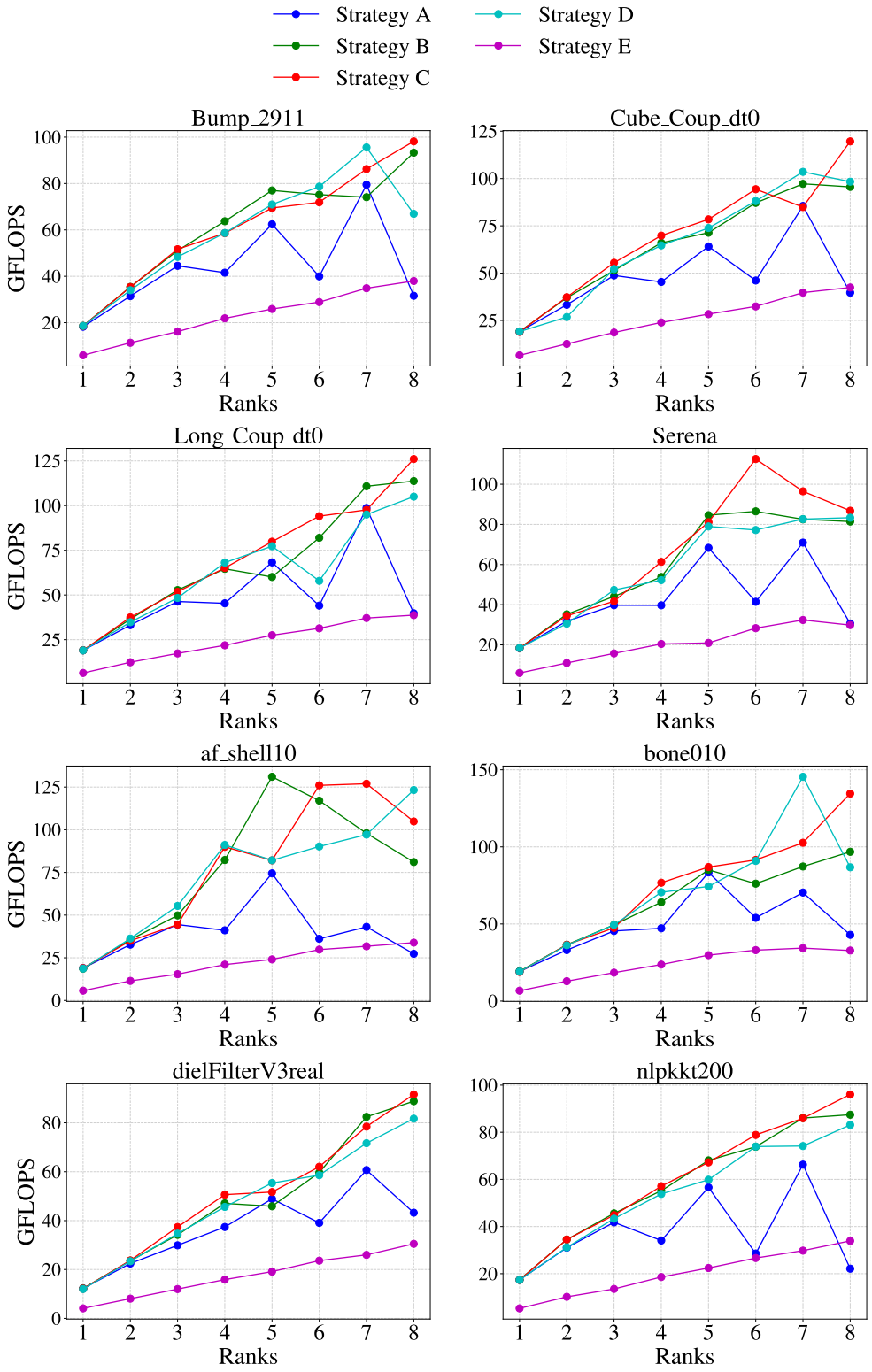


Figure 5.17: Sustained GFLOPS performance of SpMV over 100 iterations on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

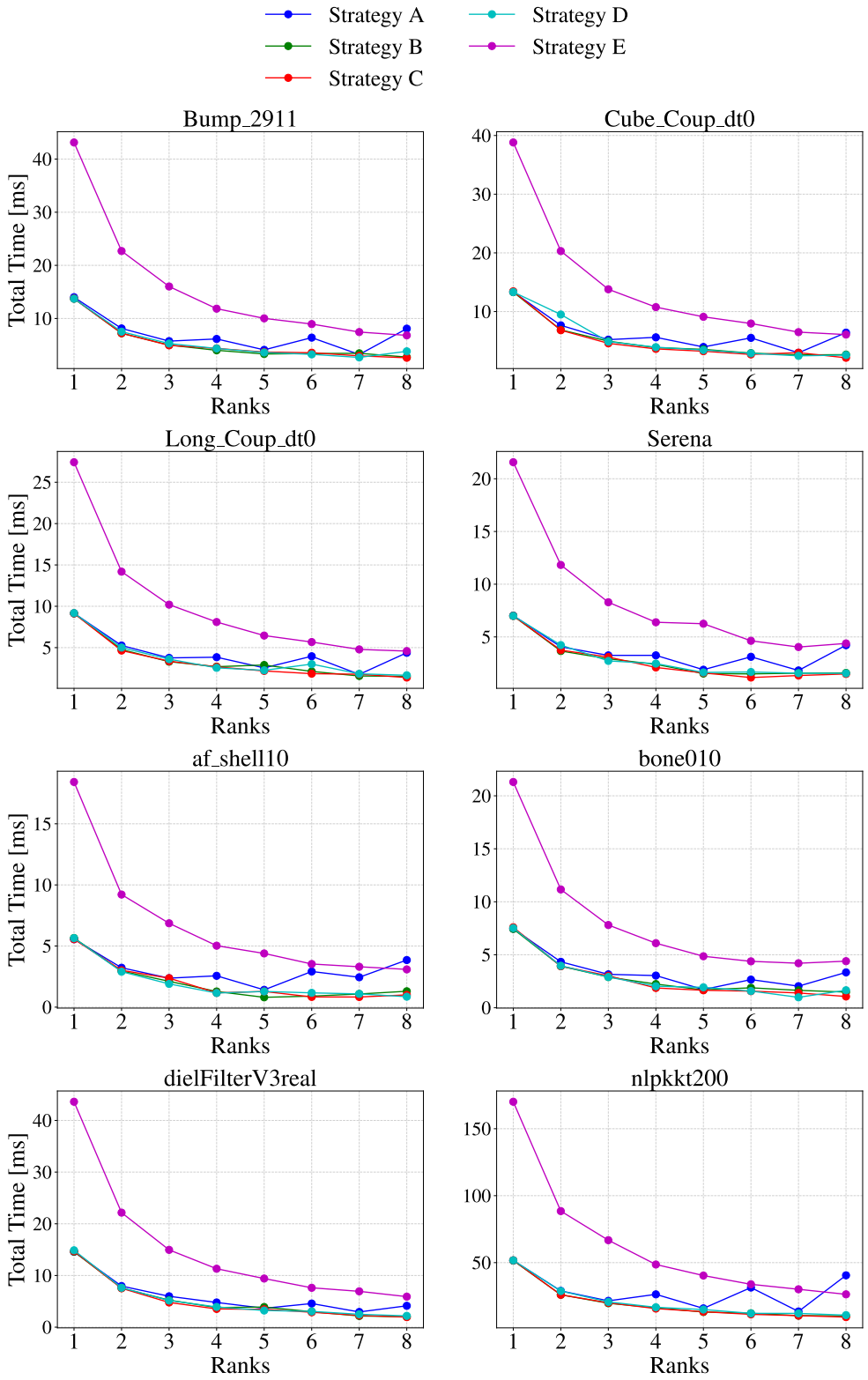


Figure 5.18: Total execution time per iteration of SpMV on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

Figure 5.17 and 5.18 presents the achieved GFLOPS and total execution time across up to eight single-socket nodes using the AMD EPYC 7302P processor, respectively. These results offers insight on the performance not only on multiple nodes, but also when the number of MPI ranks is not a power of two.

As is evident from these results, Strategy E consistently underperforms compared to the other strategies in a multi node setting, and Strategy A exhibits a sawtooth like pattern. We can however observe that Strategy E trends towards better performance, and it is possible it can catch up to the other strategies as the number of nodes increase.

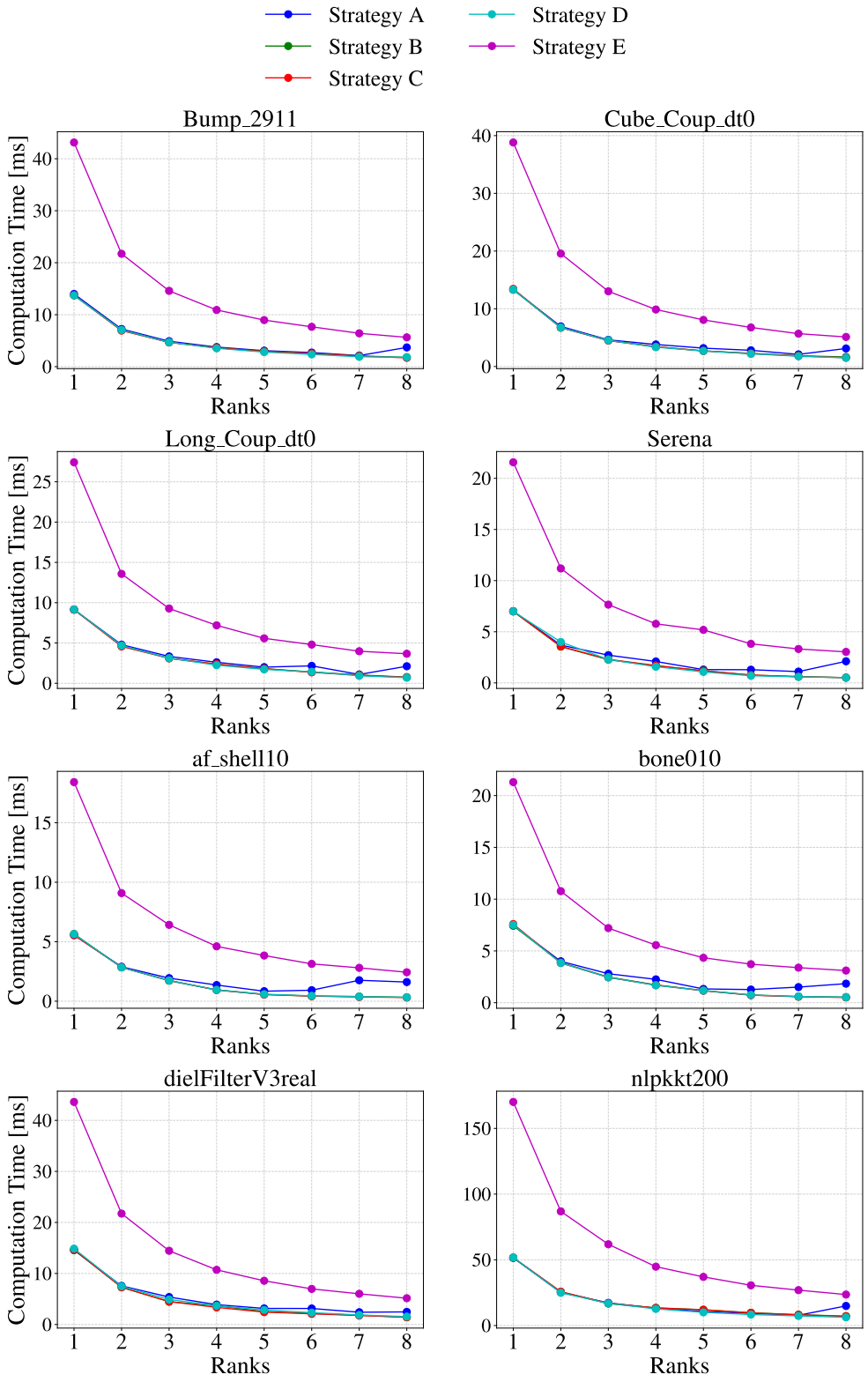


Figure 5.19: Computation time per iteration of SpMV on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

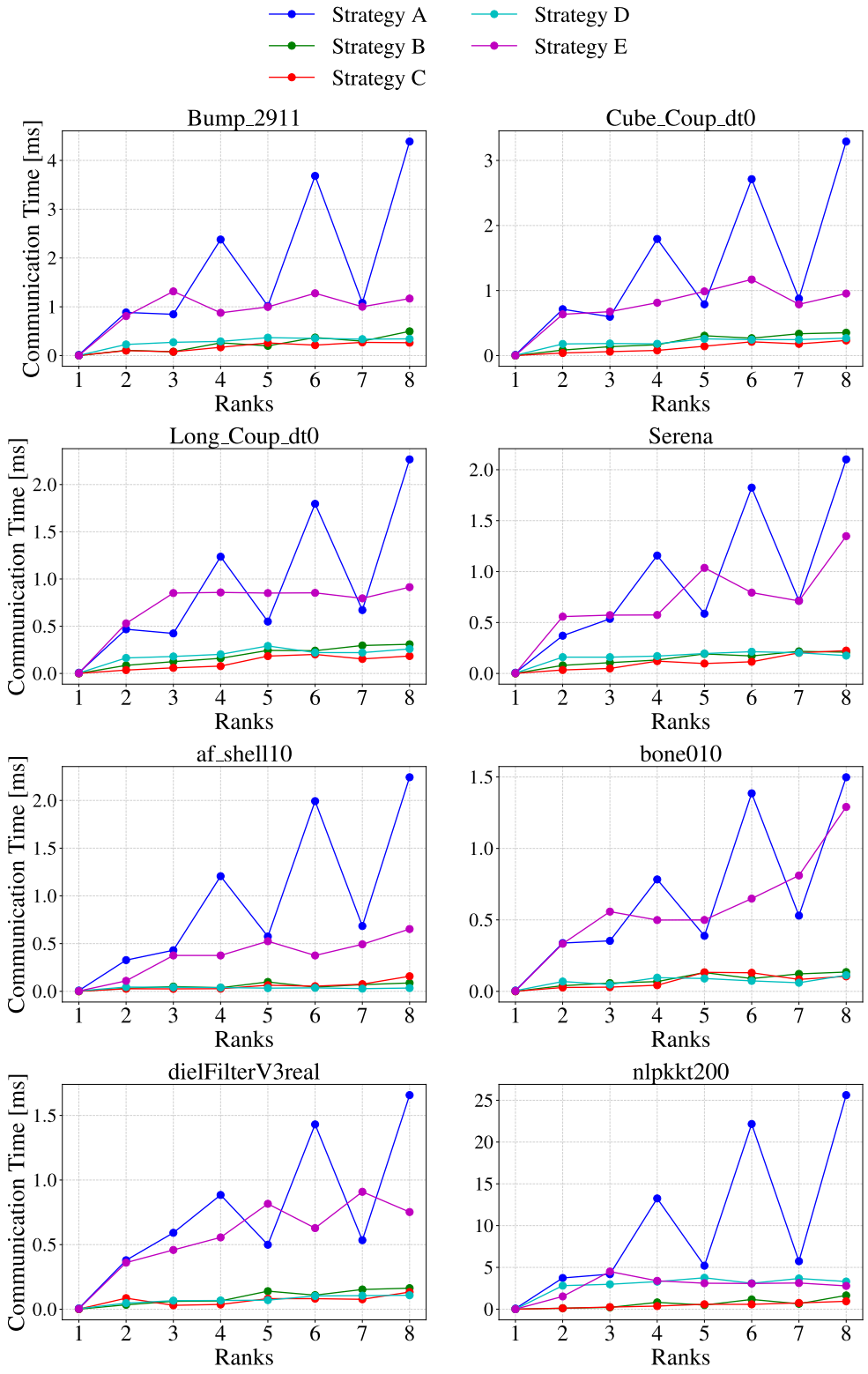


Figure 5.20: Communication time per iteration of SpMV on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

Figure 5.19 and 5.20 illustrate the computation time and communication time per iteration of SpMV. Strategy A exhibits a sawtooth-like pattern, depending on whether or not the number of nodes used is even or odd. Such irregularities is can be explained by the underlying communication algorithms MPI decides to employ when using `MPI_Allgatherv` (see [17]). MPI decides on some internal algorithm to use in collective communication routines, depending on the number of ranks involved in the communication and the size of the messages, amongst other factors. The exact algorithm MPI uses internally for these results is unknown, but it is the most probable reason that this sawtooth pattern is observed, this is also backed up by the fact that the results in Figure 5.20, don't exhibit this behaviour.

Strategy E consistently underperforms compared to the other strategies, both in computation time and in communication time, which was also observed in Figure 5.9, 5.10, 5.14 and 5.15. The reason for the underperformance in computation is once again due to the impact reordering has on the cache hit rate (see Section 5.3.1 and [18]). The reason for the underperformance in communication time is grounded in the difference of the implementation of the communication discussed in Section 4.5. Posting multiple `MPI_Send` and `MPI_Irecv` yields worse performance than posting only one `MPI_Ialltoallv` and using `MPI_Waitall` to wait for the requests to finish.

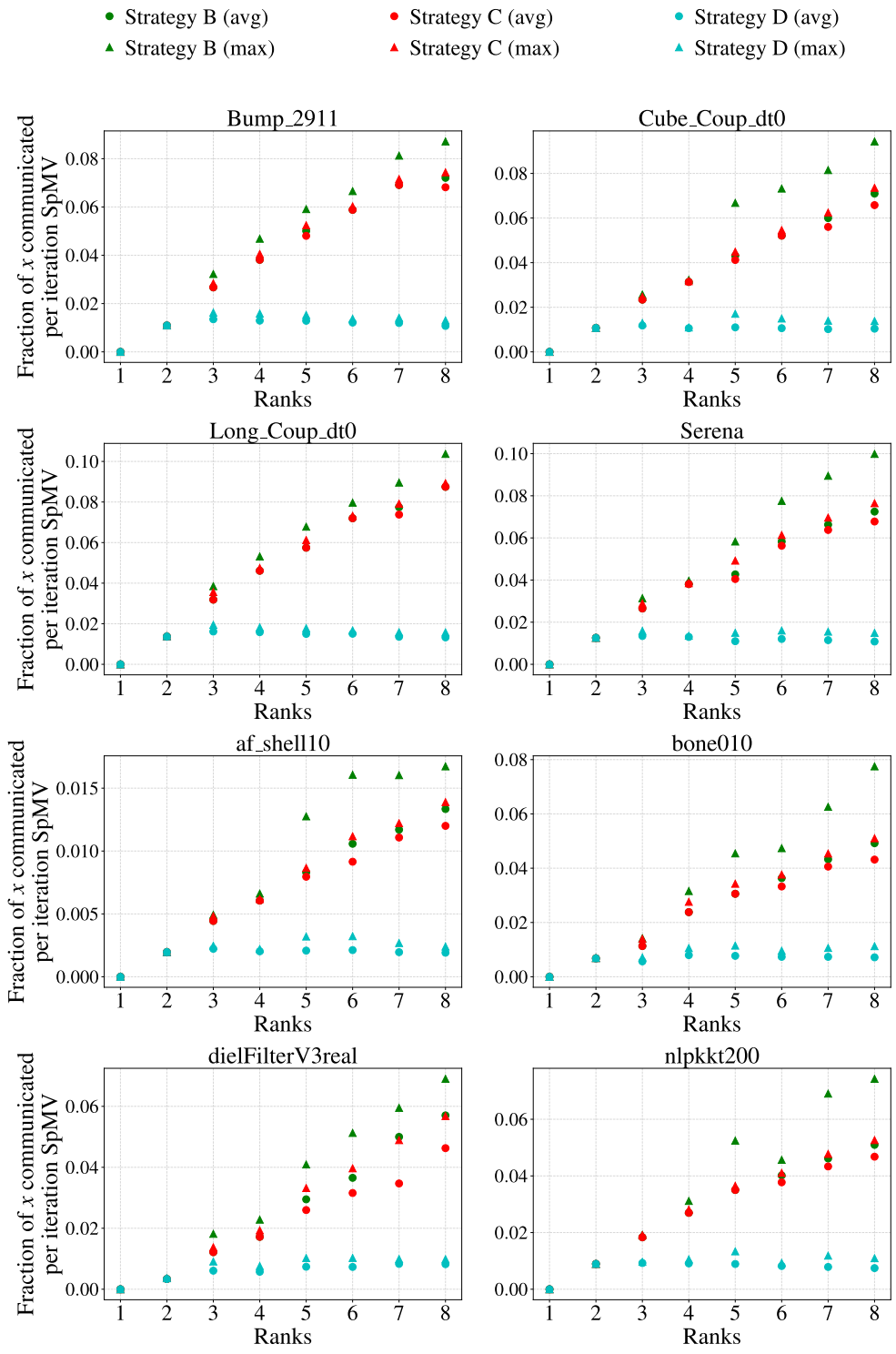


Figure 5.21: Fraction of the global x vector communicated per iteration of SpMV on 1–8 nodes (one MPI rank per node) using single-socket AMD EPYC 7302P processors.

Figure 5.21 shows the communication volume for each communication strategy per iteration of SpMV. We can see that communication volume in Strategy B and C are similar, but that the maximum load for a rank in Strategy B is consistently higher than that of Strategy C. Although this is just difference of a few percent at most, it provides numerical data as to why Strategy C outperforms Strategy B. We can also observe that the greatest difference in communication volume is $\approx 8.5\%$, and occurs on the `Long_Coup_dt0` matrix.

5.5 AMD EPYC 7413

Two sets of experiments were conducted on the AMD EPYC 7413 nodes. As these nodes are also dual-socketed, the experiments conducted are similar to those on the AMD EPYC 7601 nodes, but they only utilize 24/48 OpenMP threads per socket/node, as the AMD EPYC 7413 chip provides 24 physical cores.

The results on the AMD EPYC 7413 nodes exhibit a high degree of erraticism, and are very inconsistent across the various matrices. This is due to one of the network interface cards on one of the AMD EPYC 7413 nodes being broken. Therefore these results will only be presented, but they will not be analyzed.

One MPI rank per node

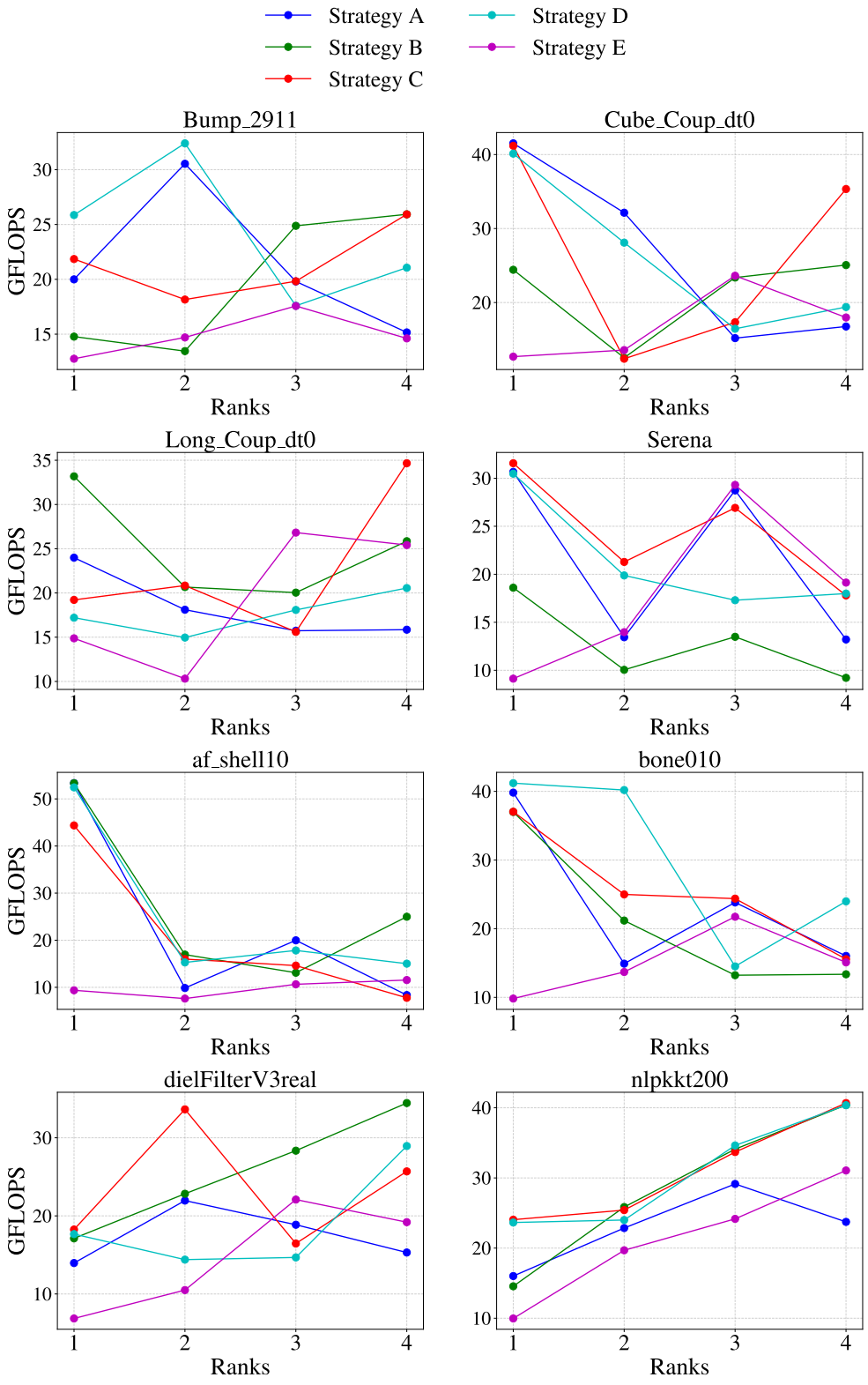


Figure 5.22: Sustained GFLOPS performance of SpMV over 100 iterations on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7413 processors.

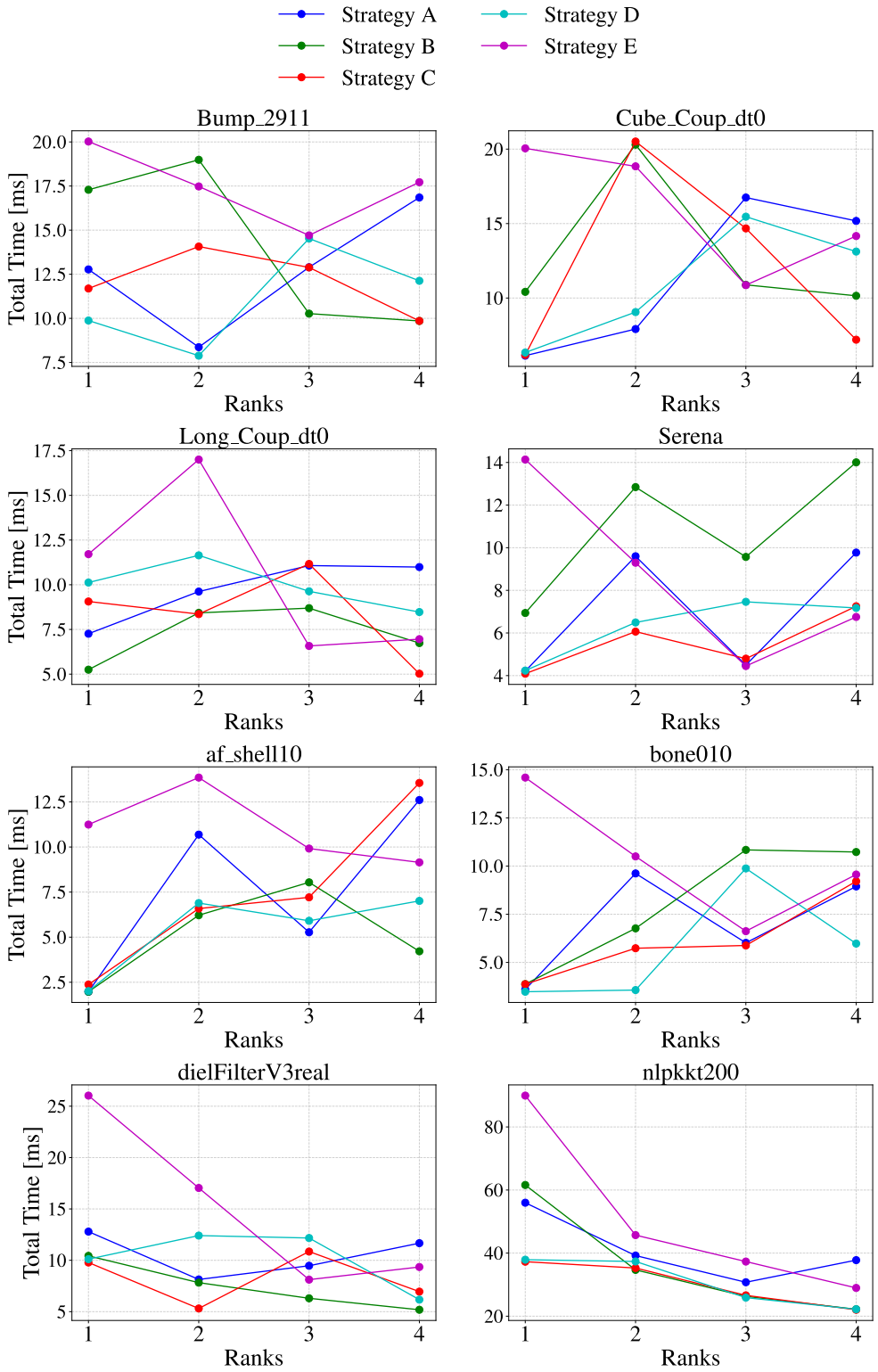


Figure 5.23: Computation time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7413 processors.

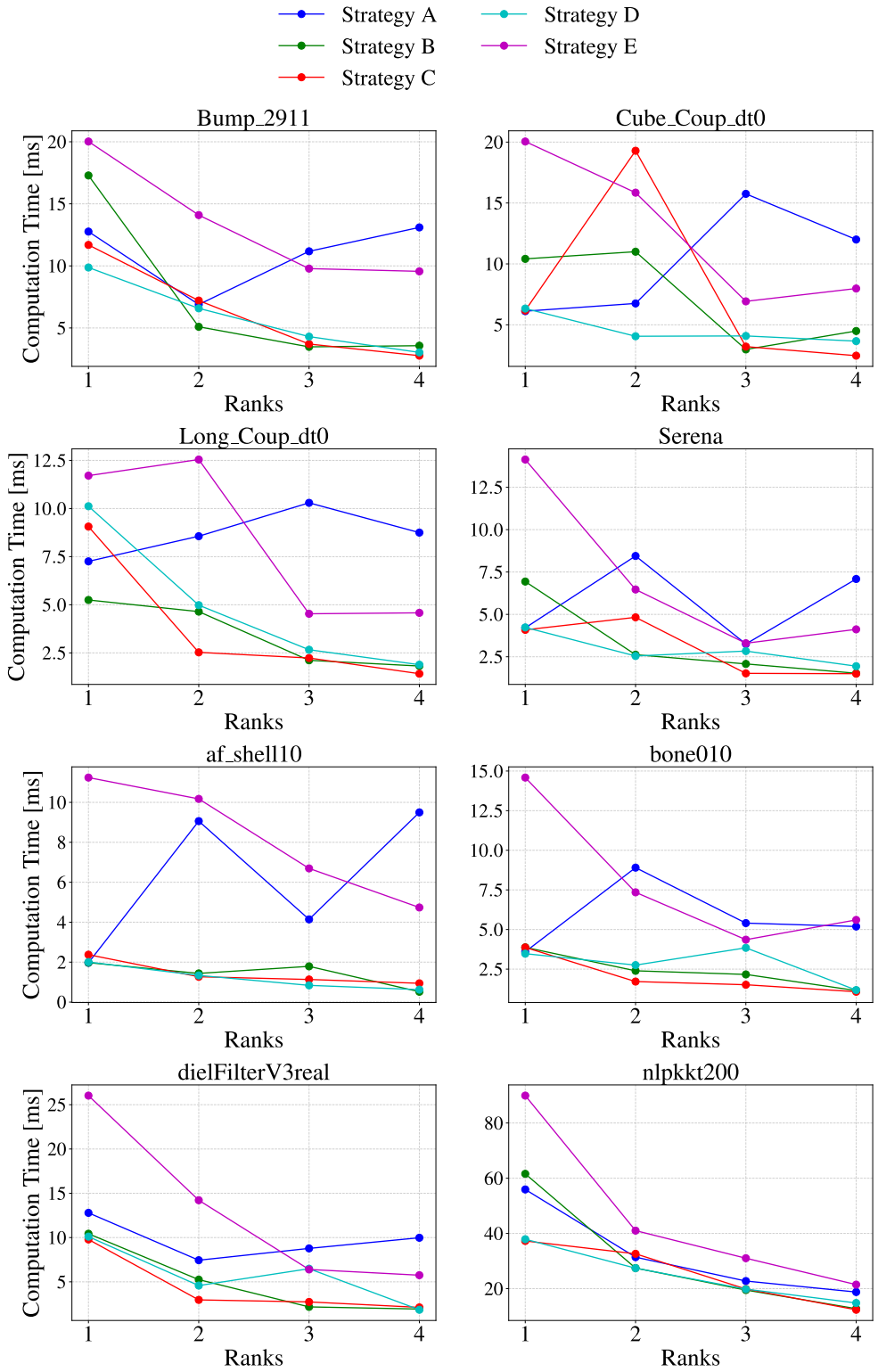


Figure 5.24: Computation time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7413 processors.

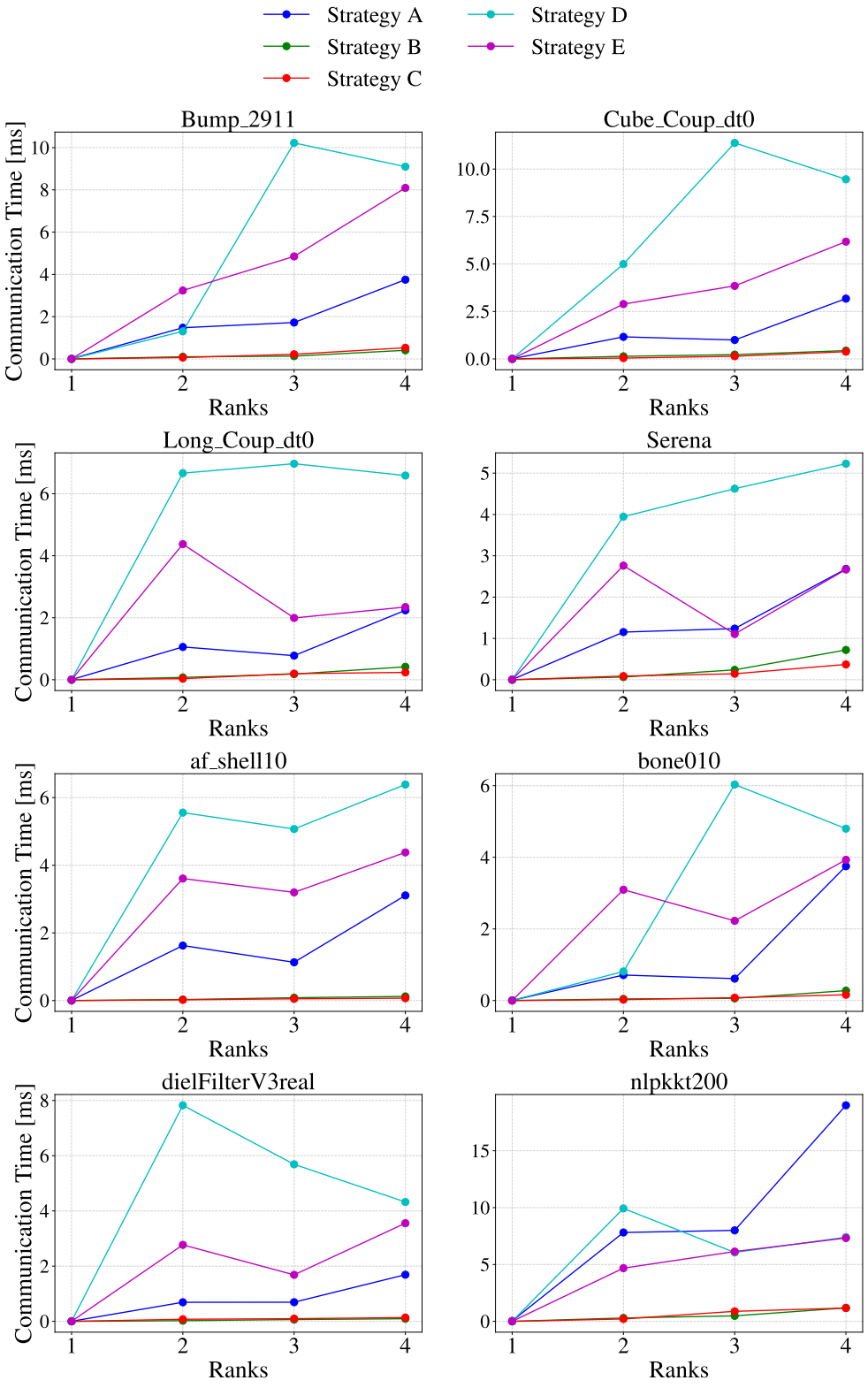


Figure 5.25: Communication time per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7413 processors.

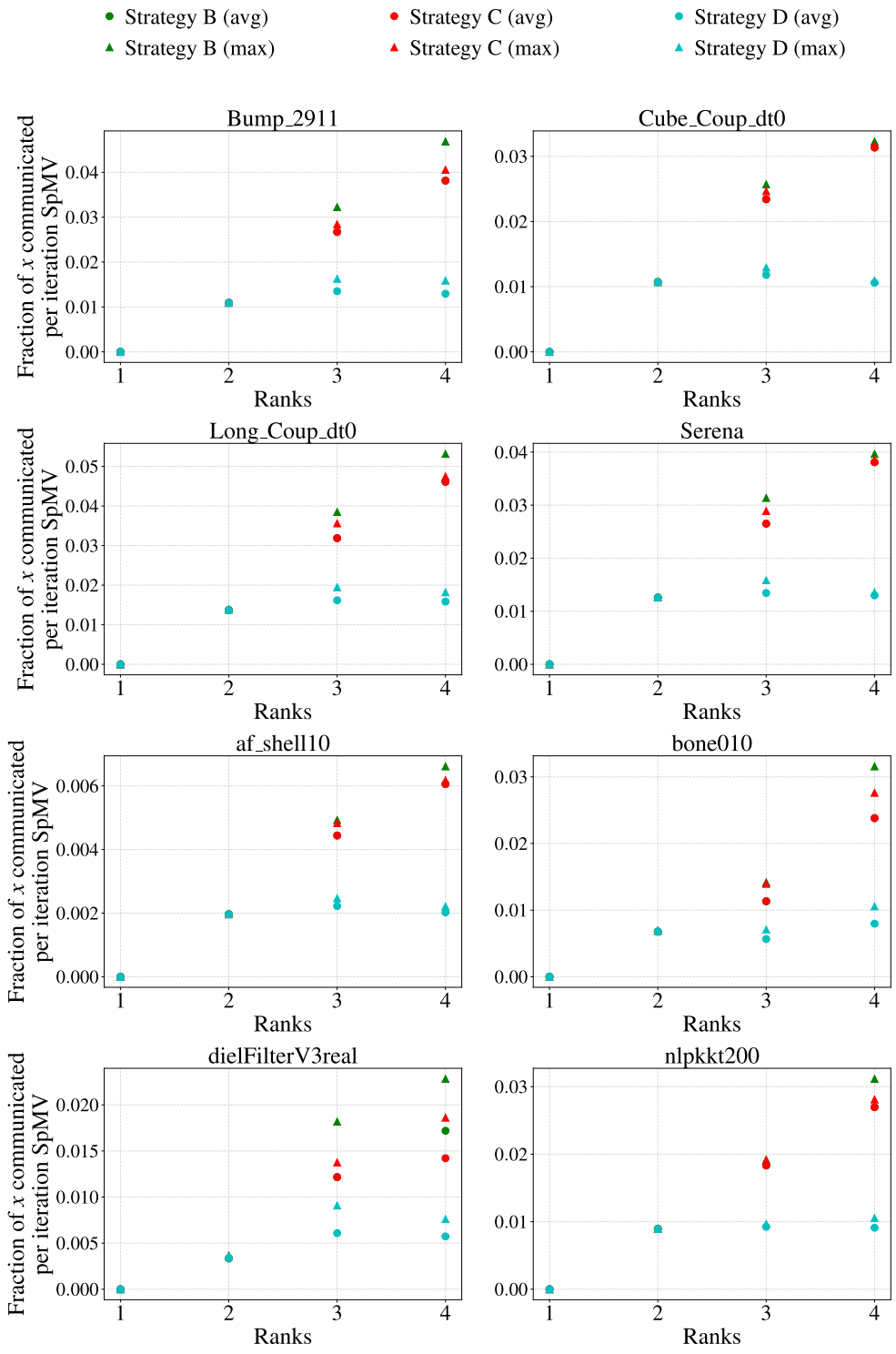


Figure 5.26: Fraction of the global x vector communicated per iteration of SpMV on 1–4 nodes (one MPI rank per node) using dual-socket AMD EPYC 7413 processors.

One MPI rank per socket

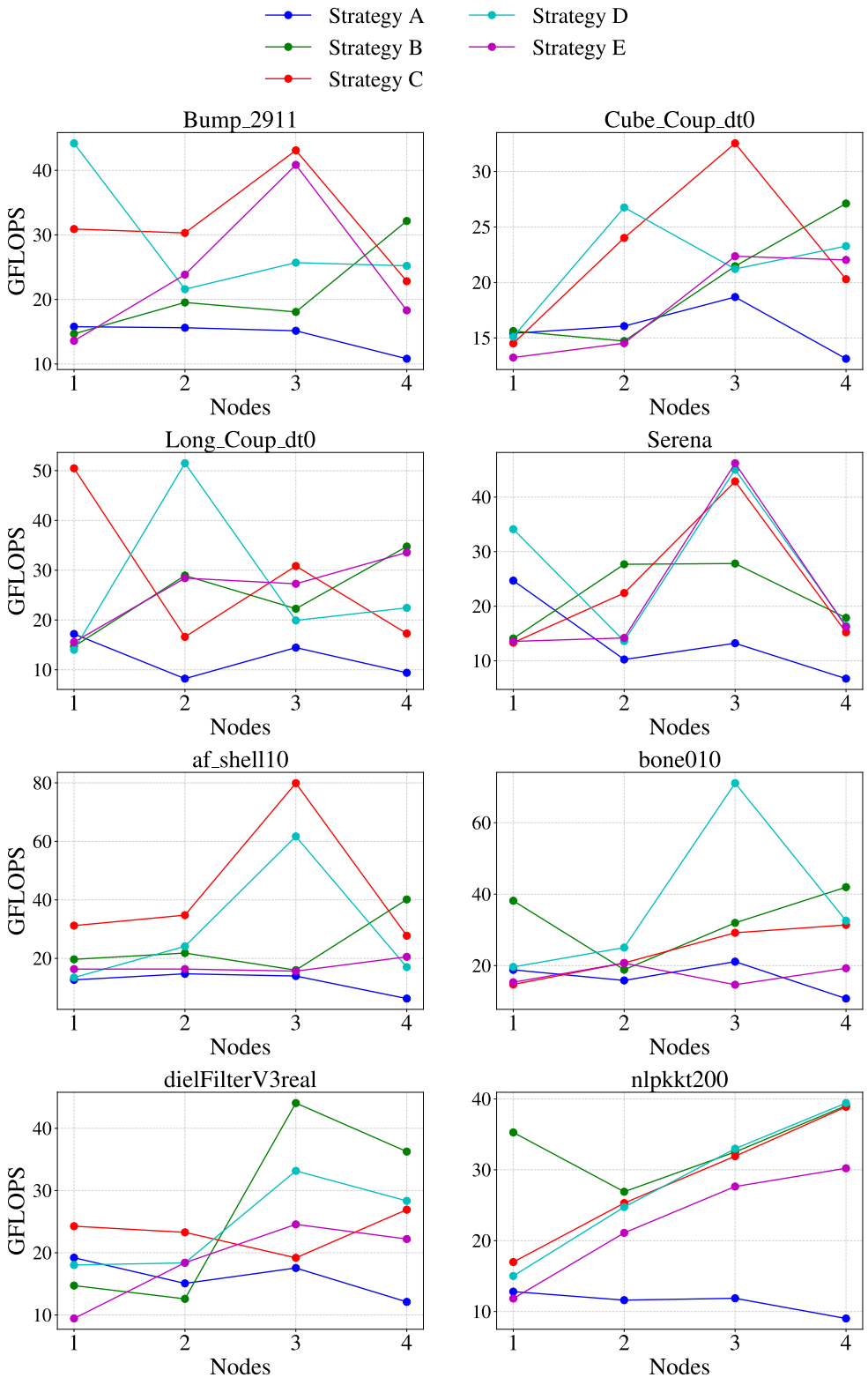


Figure 5.27: Sustained GFLOPS performance of SpMV over 100 iterations on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7413 processors.

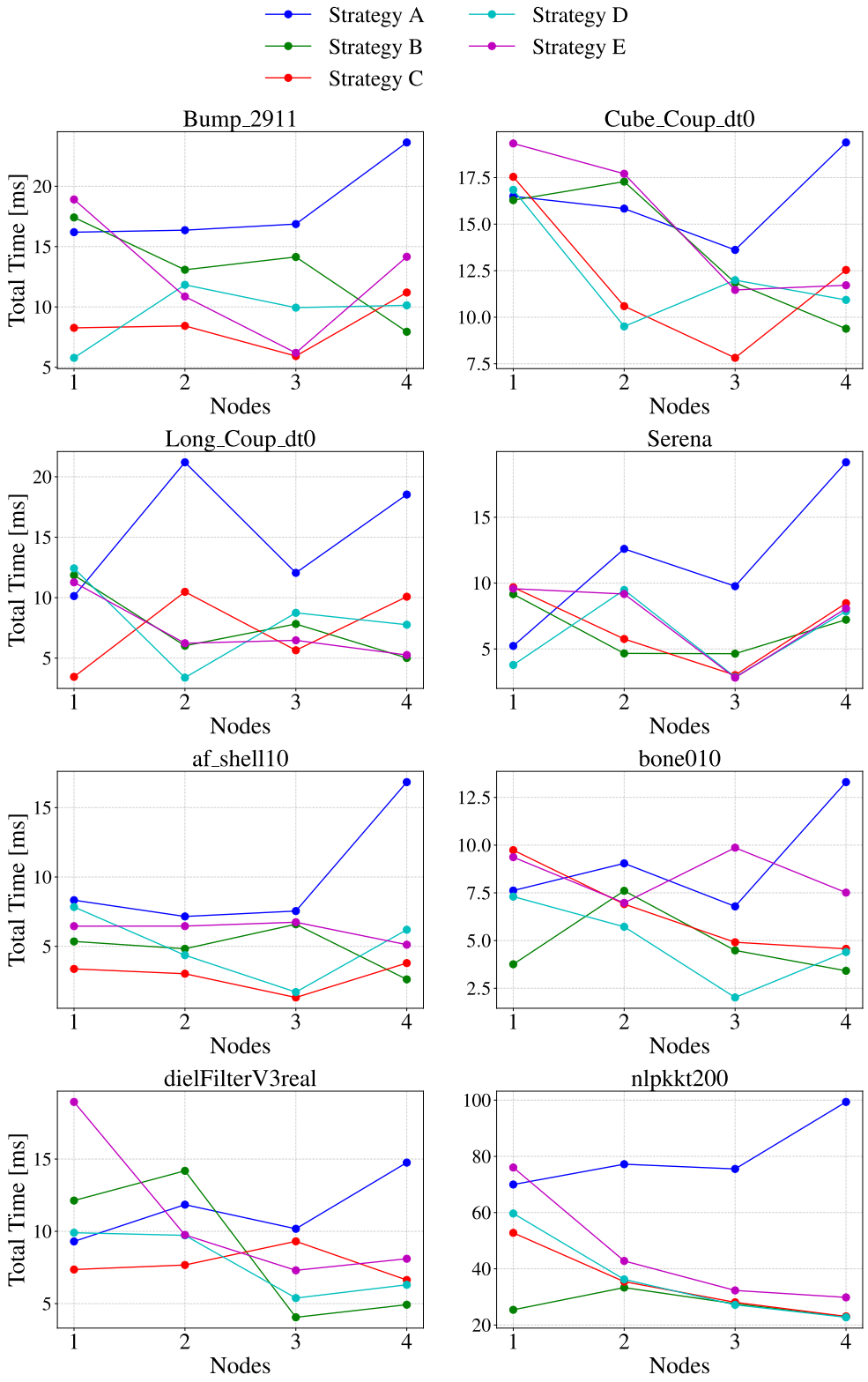


Figure 5.28: Computation time per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7413 processors.

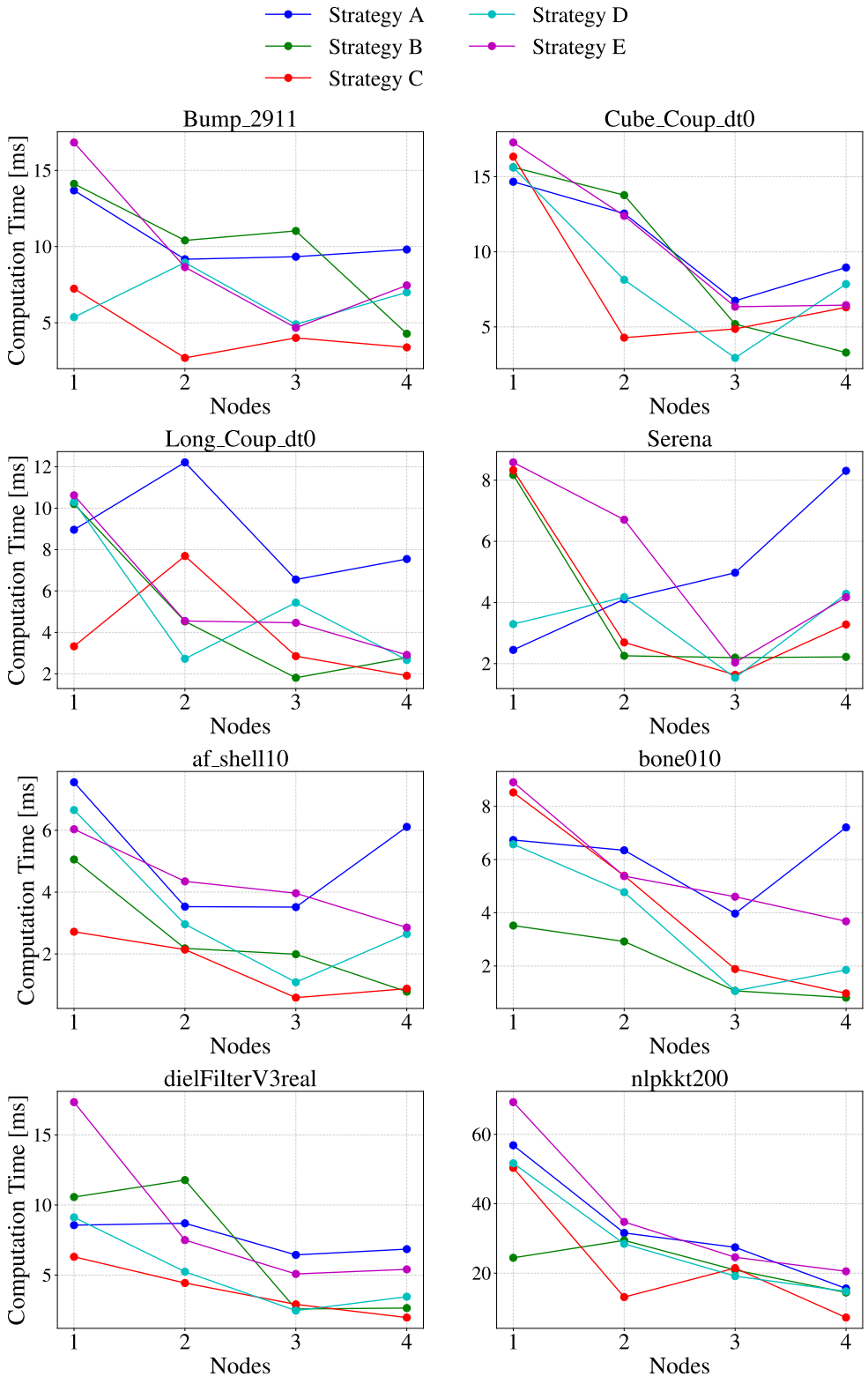


Figure 5.29: Computation time per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7413 processors.

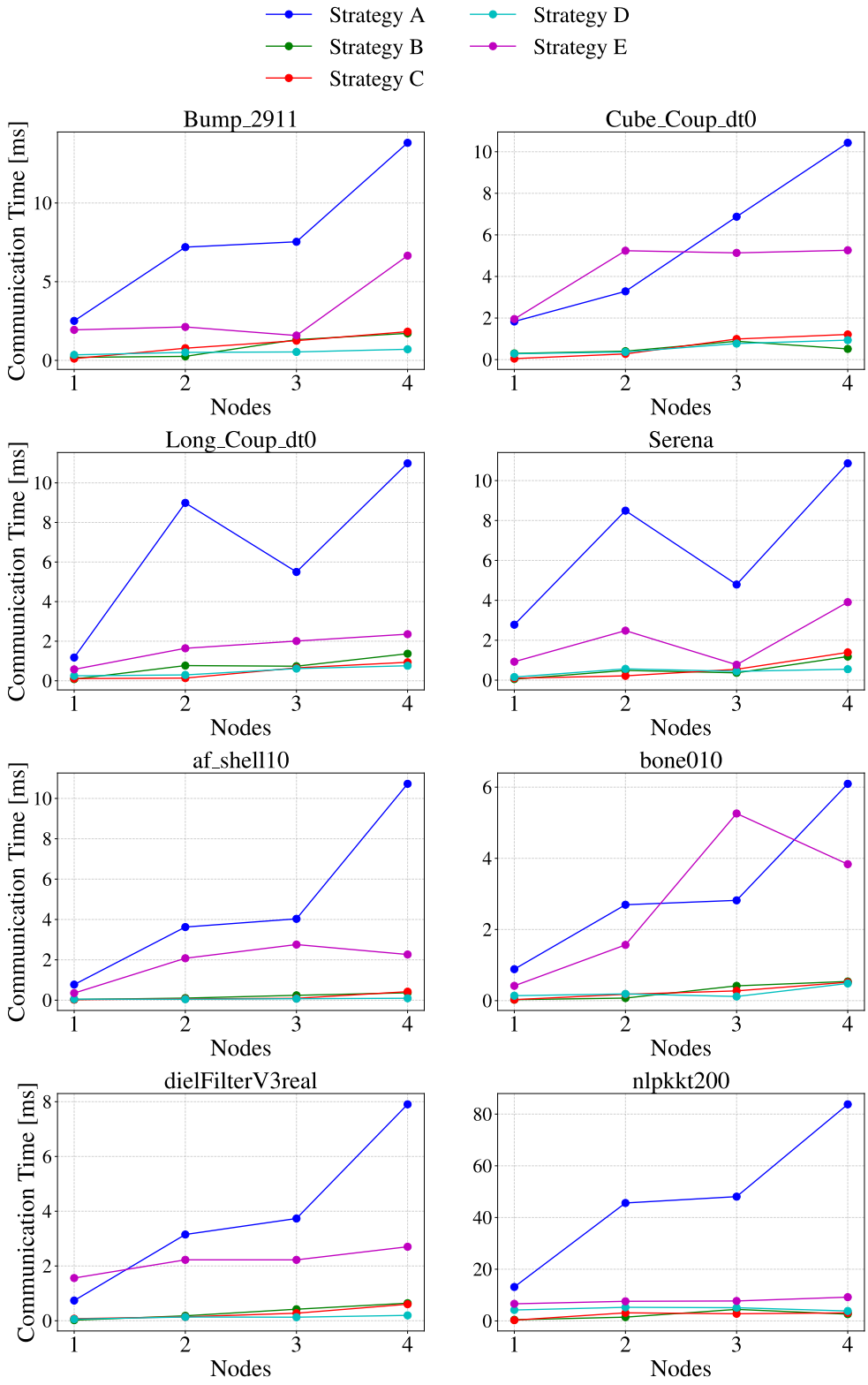


Figure 5.30: Communication time per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7413 processors.

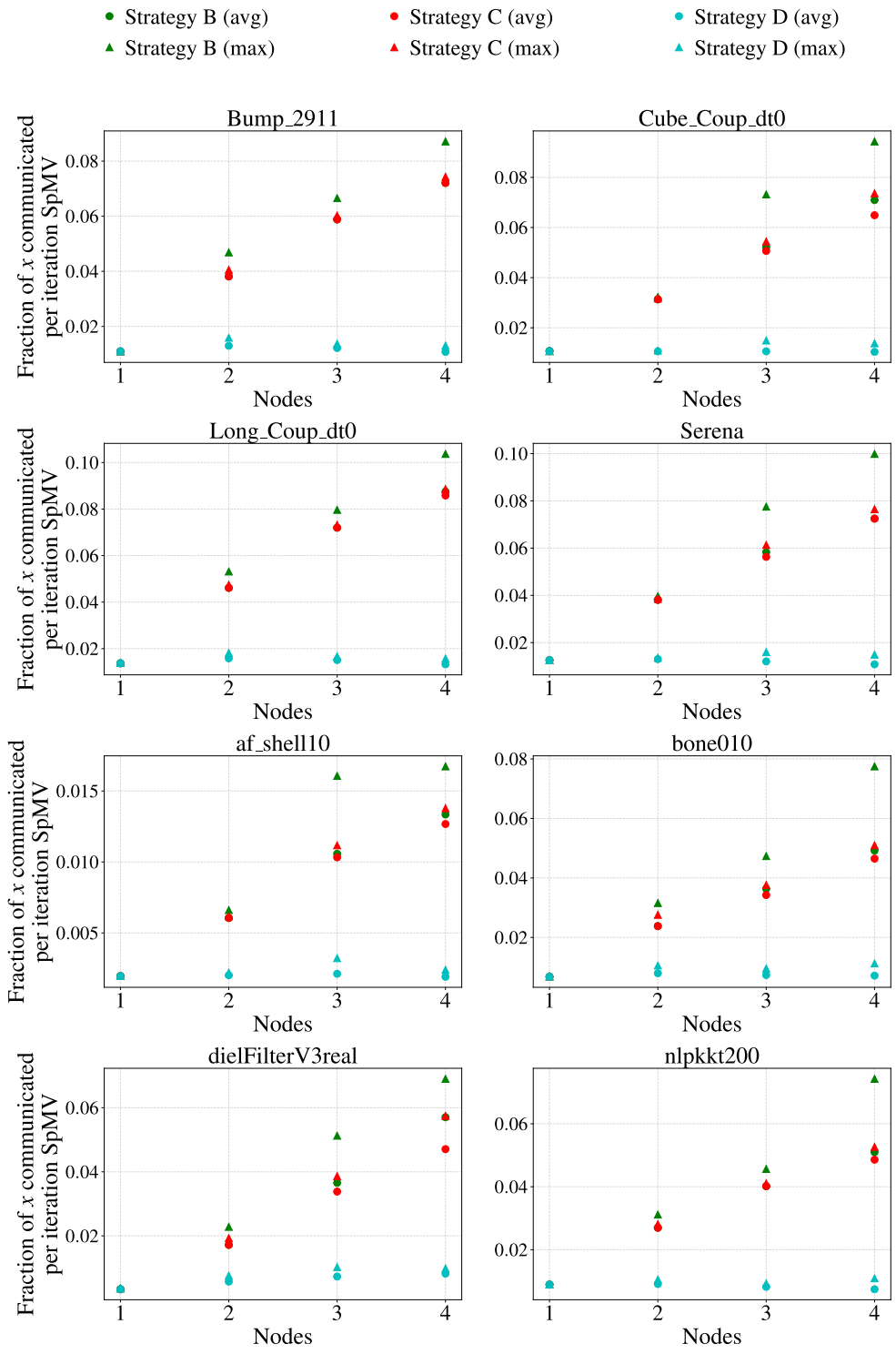


Figure 5.31: Fraction of the global x vector communicated per iteration of SpMV on 1–4 nodes (one MPI rank per socket) using dual-socket AMD EPYC 7413 processors.

Chapter 6

Related Work

SpMV is a widely researched topic within the field of high performance computing, as it is an operation which appears in a broad range of scientific domains. Efforts include the optimization of the performance of SpMV on different architectures, such as CPUs, GPUs, and most recently IPUs. Effort has also been put into developing and testing various storage formats, testing the difference between various partitioning methods, and the effect that reordering matrices have on performance.

The following paragraphs present a selection of papers that are relevant to the work done in this thesis.

Trotter et al. [18] conducted a study investigating the potential benefits of reordering matrices have on overall performance. The study aimed to find the relationship between various matrix reordering algorithms and sparse matrix operations, with particular focus on SpMV. By evaluating the performance of six different reordering algorithms, tested on 490 matrices across eight different multicore architectures, they found that graph partition based reodering yielded better performance on most matrices.

Merrill and Garland [13] conducted a study in which they present a merge-based method for CSR SpMV, that ensures strict load balancing regardless of the matrix structure, while operating directly on the CSR without preprocessing. To achieve this, they employ a merge-based parallel decompostion method on the matrix. The authors found that their decomposition method yielded highly consistent performance results. Their results matched or exceeded those of existing CSR SpMV implementation that employ other decomposition mehtods such as row-wise or nonzero-wise splitting.

Alappat et. al [1] conducted a study investigating the usage of a level-based blocking method for improving the performance of repeated SpMV. They focused their efforts on the sparse matrix power kernel, where $Ax, A^2x, A^3x, \ldots, A^kx$ is computed for a matrix A, vector x and a constant k. In this study, a recursive algebraic coloring engine was used to facilitate blocking of the sparse matrix data, as well as using breadth-first search to form levels on the data. The approach they used is structure independent, and it avoids shortcomings of existing blocking methods. They compared the results of their method to an optimal SpMV baseline, using a single multicore chip with both Intel and AMD architectures, and showed that their method yielded speedups of 3x, and 5x respectively.

Burchard et al. [2] conducted a study in which they investigated how to parallelize numerical computations involving unstructured meshes on Graphcore IPUs. They demonstrated how they used the Poplar SDK from Graphcore to port two computational kernels (one of which being SpMV) to these IPUs. In addition to this, they present the achievable performance and numerical accuracy of an ODE solver on this architecture.

Gottesbüren et. al [5] presents a scalable high-quality hypergraph partitioning framework they named Mt-KaHyPar. Their framework closes the gap between sequential and distributed memory partitioners by employing shared-memory algorithms in the components that make up the best sequential solvers. They evaluated their solver on a set of over 800 graphs and hypergraphs, and compared it to a set of 25 existing partitioning algorithms, and found that their fastest configuration outperformed most of the existing hypergraph partitioners in both solution quality and execution time. They also found that the highest quality configuration was faster than that of KaHyPar, another hypergraph partitioner (see [15]), while still producing the same quality partitions.

Chapter 7

Conclusion

In this thesis, the performance metrics of five increasingly selective communication strategies for parallel Sparse-Matrix Vector Multiplication were tested and evaluated. Eight well-structured sparse matrices were used in the test set, and they were tested with a set of parallel configurations on three different compute nodes, each with different architectures. These experiments were conducted in order to evaluate the communication volume of each strategy, and the effect the communication volume has on the overall performance of SpMV.

The findings show that the communication volume of the strategies were decreased as the strategies became more selective. In the single-node setting, Strategy D and E, which employ the most selective communication pattern, yielded the best results. Between these two, Strategy D consistently outperforms Strategy E, as it omits matrix reordering, thus preserving the better cache hit rate that are often encountered in well-structured matrices.

On configurations using multiple compute nodes, the results illustrate more erratic performance characteristics. The findings show that when accessing the network cards connecting the compute nodes, Strategy B and C can outperform D and E, even though their communication volume is greater. Another trend in the multi-node experiments is that Strategy E is consistently outperformed in communication time. As mentioned, the reason for this the custom implementation of all to all communication used by Strategy E.

The findings show that optimal SpMV performance in a multi-node environment is not achieved solely by employing the usage of communication minimal and/or memory-minmal strategies. SpMV performance is shown to be impacted by a number of factors, including cache hit rate, load

balancing, memory bandwidth, and communication volume. Moreover, exactly which communication strategy performs the best is dependant on the size and structure of the matrix, as well as the specifications and configurations of the hardware that is used. There is no one size fits all approach for achieving the best possible performance, but having a set of communication strategies to pick from is beneficial, as it allows the user to pick the best performing strategy for a given matrix.

This thesis has only covered the communication strategies on a set of small to medium sized sparse matrices, most of which are well-structured. It would be insightful to perform experiments that included a larger set of matrices with more diverse structures. It would also be insightful to perform experiments on larger machines, employing the usage of a greater number of compute nodes. Porting the communication strategies to other architectures, such as GPUs and IPUs would also be interesting, even though many of the same challenges will be encountered on these architectures as well.

Chapter 8

Appendix

The full source code for the results produced for communication Strategy A-D in this thesis is publicallly available on GitHub:

- Repository: github.com/kristiansordal/SPMV.
- Commit Hash: `44ac5e5589a71051afca2740aba8245fc3a86d67`

This code was adapted from the original work by Kenneth Langedal, available publically on GitHub:

- Repository: github.com/KennethLangedal/INF339
- Commit Hash: `da00d109e78c7f7a3ad560b9917e52daaaa94b19`

The source code for Strategy E was adapted from Trotter et. al [19]. The code is available publically on GitHub:

- Repository: github.com/ParCoreLab/aCG/
- Commit Hash: `7a472381449d38e393ba1e0f0d2addcdc3155869`

Bibliography

[1] Christie Alappat, Georg Hager, Olaf Schenk, and Gerhard Wellein. Level-based blocking for sparse matrices: Sparse matrix-power-vector multiplication. *IEEE Transactions on Parallel and Distributed Systems*, 34(2):581–597, 2023.

[2] Luk Burchard, Kristian Gregorius Hustad, Johannes Langguth, and Xing Cai. Enabling unstructured-mesh computation on massively tiled ai processors: An example of accelerating in silico cardiac simulation. *Frontiers in Physics*, Volume 11 - 2023, 2023.

[3] U.V. Catalyurek and C. Aykanat. Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication. *IEEE Transactions on Parallel and Distributed Systems*, 10(7):673–693, 1999.

[4] Timothy A. Davis and Yifan Hu. The university of florida sparse matrix collection. *ACM Trans. Math. Softw.*, 38(1), December 2011.

[5] Lars Gottesbüren, Tobias Heuer, Nikolai Maas, Peter Sanders, and Sebastian Schlag. Scalable high-quality hypergraph partitioning. *ACM Trans. Algorithms*, 20(1), January 2024.

[6] Gautam Gupta, Sivasankaran Rajamanickam, and Erik G. Boman. GAMGI: Communication-reducing algebraic multigrid for gpus. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 61–75. ACM, 2024.

[7] Cerebras Systems Inc. Wse-3 datasheet. https://cerebras.net/product/system/, 2024. DS03 v3 821.

[8] George Karypis and Vipin Kumar. METIS: A Software Package for Partitioning Unstructured Graphs, Partitioning Meshes, and Computing Fill-Reducing Orderings of Sparse Matrices. Technical

report, University of Minnesota, Department of Computer Science / Army HPC Research Center, 1997. Version 3.0.3.

[9] Scott P. Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. The suitesparse matrix collection website interface. *Journal of Open Source Software*, 4(35):1244, 2019.

[10] Christoph Lameter. Numa (non-uniform memory access): An overview: Numa becomes more common because memory controllers get close to execution units on microprocessors. *Queue*, 11(7):40–51, July 2013.

[11] Lawrence Livermore National Laboratory. Introduction to parallel computing tutorial. http://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial, 2024. Accessed: 2025-05-08.

[12] Nakul Manchanda and Karan Anand. Non-uniform memory access (numa). *New York University*, 4, 2010.

[13] Duane Merrill and Michael Garland. Merge-based parallel sparse matrix-vector multiplication. In *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 678–689, 2016.

[14] Peter Norvig. Latency numbers every programmer should know. https://norvig.com/21-days.html#Latency, 2021. Accessed: 2025-04-29.

[15] Sebastian Schlag, Tobias Heuer, Lars Gottesbüren, Yaroslav Akhremtsev, Christian Schulz, and Peter Sanders. High-quality hypergraph partitioning, 2021.

[16] P. Stenstrom. A survey of cache coherence schemes for multiprocessors. *Computer*, 23(6):12–24, 1990.

[17] Andreas Thune, Sven-Arne Reinemo, Tor Skeie, and Xing Cai. Detailed modeling of heterogeneous and contention-constrained point-to-point mpi communication. *IEEE Transactions on Parallel and Distributed Systems*, 34(5):1580–1593, 2023.

[18] James D. Trotter, Sinan Ekmekçibaşı, Johannes Langguth, Tugba Torun, Emre Düzakın, Aleksandar Ilic, and Didem Unat. Bringing

order to sparsity: A sparse matrix reordering study on multicore cpus. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '23, New York, NY, USA, 2023. Association for Computing Machinery.

[19] James D. Trotter, Sinan Ekmekçibaşı, Johannes Langguth, Doğan Sağbili, Xing Cai, and Didem Unat. Artifacts for "CPU- and GPU-initiated Communication Strategies for Conjugate Gradient Methods on Large GPU Clusters". https://doi.org/10.5281/zenodo.15291898, April 2025. Under review.

[20] Cong Zheng, Shuo Gu, Tong-Xiang Gu, Bing Yang, and Xing-Ping Liu. Biell: A bisection ellpack-based storage format for optimizing spmv on gpus. *Journal of Parallel and Distributed Computing*, 74(7):2639–2647, 2014. Special Issue on Perspectives on Parallel and Distributed Processing.